Highlights

CAM-LDS: Cyber Attack Manifestations for Automatic Interpretation of System Logs and Security Alerts

Max Landauer, Wolfgang Hotwagner, Thorina Boenke, Florian Skopik, Markus Wurzenberger

- Public labeled log data sets of attack traces and artifacts.
- Analysis and categorization of cyber attack manifestations.
- LLM-based interpretation of system logs and security alerts.

CAM-LDS: Cyber Attack Manifestations for Automatic Interpretation of System Logs and Security Alerts

Max Landauer, Wolfgang Hotwagner, Thorina Boenke, Florian Skopik, Markus Wurzenberger

Austrian Institute of Technology, Center for Digital Safety & Security, Vienna, 1210, Austria

Abstract

Log data are essential for intrusion detection and forensic investigations. However, manual log analysis is tedious due to high data volumes, heterogeneous event formats, and unstructured messages. Even though many automated methods for log analysis exist, they usually still rely on domain-specific configurations such as expert-defined detection rules, handcrafted log parsers, or manual feature-engineering. Crucially, the level of automation of conventional methods is limited due to their inability to semantically understand logs and explain their underlying causes. In contrast, Large Language Models enable domain- and format-agnostic interpretation of system logs and security alerts. Unfortunately, research on this topic remains challenging, because publicly available and labeled data sets covering a broad range of attack techniques are scarce. To address this gap, we introduce the Cyber Attack Manifestation Log Data Set (CAM-LDS), comprising seven attack scenarios that cover 81 distinct techniques across 13 tactics and collected from 18 distinct sources within a fully open-source and reproducible test environment. We extract log events that directly result from attack executions to facilitate analysis of manifestations concerning command observability, event frequencies, performance metrics, and intrusion detection alerts. We further present an illustrative case study utilizing an LLM to process the CAM-LDS. The results indicate that correct attack techniques are predicted perfectly for approximately one third of attack steps and adequately for another third, highlighting the potential of LLM-based log interpretation and utility of our data set.

Keywords: system log data, cyber attack data set, log interpretation, large language models, intrusion detection, security alerts

1. Introduction

Protection against the ever-growing increase of cyber attacks is of paramount importance for organizations across all sectors. Among the various domains of cyber security, log data analysis stands out as both highly valuable and inherently complex. On the one hand, system log data presents a persistent and fine-grained record of activities occurring on networks and computer systems. Accordingly, they constitute a fundamental source of information for the detection of immanent attacks as well as forensic investigations of malicious or undesired activities after the fact [1].

On the other hand, log data appear in large volumes, involve heterogeneous formats, and often contain unstructured messages. Their interpretation therefore requires substantial manual effort and expert knowledge [2, 3, 4]. Although automated analysis methods exist, their practical applicability remains limited: signature-based detection approaches rely on manually crafted rules that are difficult to maintain and ineffective against novel and unknown attacks [5]; anomaly-based methods require large amounts of representative training data [6], frequently suffer from high false positive rates [5], depend on manual feature engineering [7], often struggle to adapt to evolving attack patterns [7], and typically report statistical deviations without providing semantic explanations of the underlying causes [8, 6, 3].

The emergence of Large Language Models (LLM) has significantly changed how security engineers approach log analysis. Recent studies in Security Operations Centers (SOC) indicate that practitioners increasingly use LLMs to interpret and explain system logs and other low-level telemetry data [9]. Thereby, the key capability of LLMs is their ability to semantically analyze log data in a manner analogous to natural language [10], which supports engineers in understanding unfamiliar log formats or unknown events without manual lookup of documentations [11].

Aside from direct interpretation of log snippets, LLMs have been successfully demonstrated for various log analysis tasks such as log parsing, failure detection, intrusion detection, root cause analysis, and summarization [12, 13, 14]. In contrast to traditional approaches that require separate specialized models for different tasks and log formats, LLMs provide a unified framework capable of supporting or autonomously performing multiple

analysis objectives [13, 10]. Consequently, the automation of SOC workflows through LLM-based methods is expected to expand further, even though several challenges remain, including significant computational overhead, high costs, hallucinations, inconsistent outputs, ambiguous terminology, oversimplified reasoning, and limited transparency [15].

A major obstacle for advancing research on LLM-based log interpretation is the lack of suitable public log data sets for evaluation [15]. Researchers therefore often resort to private data sets [4, 16], which hinder reproducibility and comparability, or data sets without security context [11, 13, 10], which limits the relevance of results for security applications. While some log data sets from the intrusion detection domain exist, they typically focus on network traffic [17, 18, 19], are restricted to Windows environments [20, 21, 22, 23], or originate from noisy environments in which isolating and labeling attack traces is challenging [24, 25]. Critically, most security-related data sets cover only a limited set of attack tactics and techniques, thereby failing to represent the diversity of real-world kill chains and limiting the scope of evaluations.

To address these challenges and enable reproducible research in log interpretation and related areas, we introduce the *Cyber Attack Manifestation Log Data Set (CAM-LDS)*, a collection of system log data and security alerts that are direct consequences of cyber attacks. In contrast to prior data set collection strategies, we specifically focus on implementing a broad range of distinct attack techniques and arranging them into multiple coherent kill chains. Thereby, we pursue realistic attack execution within an idle network, allowing us to collect labeled manifestations of attacker behavior while minimizing unrelated background noise. To the best of our knowledge, CAM-LDS is the first publicly available data set specifically designed to support research on LLM-based interpretation of system logs and security alerts.

We release the system log data[1] and network packet captures[2] online. In addition, we provide the virtual testbed infrastructure *AttackBed* together with all implemented attack automation scripts as open-source code[3]. Furthermore, we publish the artifacts required to reproduce our case study for LLM-based log interpretation, including prompts and model responses[4]. We

[1] `https://zenodo.org/records/18390560`
[2] `https://zenodo.org/records/18701094`
[3] `https://github.com/ait-testbed/attackbed`
[4] `https://github.com/ait-aecid/attack-manifestations-interpretation`

summarize our contributions as follows.

- A labeled data set of cyber attack manifestations comprising system logs and security alerts,
- a systematic analysis and categorization of the collected attack manifestations, and
- an illustrative evaluation of LLM-based log interpretation using the proposed data set.

The remainder of this paper is structured as follows. Section 2 summarizes related work on LLM-based interpretation of system logs and security alerts and furthermore reviews existing data sets. Section 3 describes the methodology for generating the data sets introduced in this paper. Section 4 analyzes the captured attack manifestations extracted from the data. Section 5 presents an illustrative evaluation of LLM-based interpretation. Section 6 discusses the implications and limitations of our work and outlines directions for future research. Finally, Section 7 concludes the paper.

2. Related Work

In this section we first summarize relevant publications in the area of system log and alert interpretation using LLMs. Subsequently, we provide a review of public data sets that are useful to evaluate such approaches.

2.1. LLM-based Interpretation

We first discuss approaches that apply LLMs to generic system logs and then focus on work conducted in the cyber security domain.

2.1.1. Analysis of Generic Log Data

The rapid emergence and advancement of LLMs have influenced a wide range of research fields, including system log analysis. Since logs are typically available in unstructured or highly heterogeneous formats, LLMs appear as a natural candidate to assist human analysts in extracting actionable insights from such data. One of the earliest systematic investigations in this area is presented by Mudgal et al. [12], who evaluate the extent to which LLMs can infer information directly from raw log data. They consider the following

key analysis tasks: *Log Parsing*, which aims to derive log templates and extract parameters, *Log Analytics*, which involves disciplines such as anomaly detection, root cause analysis, and event prediction, and *Log Summarization*, which provides concise summaries of large amounts of log data. In addition to parsing and anomaly detection, the study of Liu et al. [13] considers *Log Interpretation* as the generation of natural-language explanations of the semantic meaning of log messages, as well as *Root Cause Analysis* and *Solution Recommendation* as post-detection activities that aim to support analysts with actionable advice for the resolution of potential issues.

Several recent publications present approaches that automate some or all of these log analysis tasks. Liu et al. [6] propose LogPrompt, a framework for anomaly detection in log data using tailored prompts for ChatGPT. In their subsequent work, the authors propose LogLM [13], a model designed to support all major log analysis tasks within a unified framework. Shanto et al. [4] develop a domain-specific LLM trained explicitly to explain error messages in console logs. Qi et al. [2] introduce LogGPT, which leverages ChatGPT to detect log anomalies in a zero-shot setting, where no training data is provided, as well as a few-shot setting, where labeled sample data is available to the model. While many studies primarily evaluate detection accuracy, Zhang et al. [3] emphasize the practical importance of providing meaningful interpretations alongside detections. To address this gap, they fine-tune a model termed LLM-LADE, which jointly performs anomaly detection and explanation.

2.1.2. Analysis of Security Logs and Alerts

All approaches discussed in the previous section are evaluated on data sets that comprise log data corresponding to normal system or user behavior as well as failure or errors logs. A key difference to log data analysis in the security context is that data sets comprise attack traces that result from deliberate and adversarial actions rather than unintentional failures. While some concepts from anomaly detection and interpretation are applicable in both domains, correct classification and reasoning about attack manifestations generally requires to understand an attacker's intent, place each attack step in the context of an attack chain, and incorporate external knowledge from attack frameworks and tools.

Karlsen et al. [10] evaluate LLMs on both generic and security-focused data sets and show that fine-tuning substantially improves detection performance across domains. Additionally, they demonstrate how LLMs can

highlight those parts of log entries that contribute most strongly to a classification decision, thereby enhancing model transparency for human analysts. Palma et al. [7] address the high computational and financial costs associated with large proprietary models and instead evaluate a locally deployed language model for log-based detection and interpretation. Sun et al. [26] present a framework for host-based intrusion detection that relies on analysis of raw system logs by LLMs. Their work addresses common issues for such tasks, such as context window limitations that prevent analysis of large chunks of log data.

Beyond log explanation, LLMs have also been applied to map low-level log data to higher-level threat intelligence concepts. Cotti et al. [16] present OntoLogX, which constructs ontology-grounded knowledge graphs from raw logs and uses them to predict MITRE ATT&CK tactics. Tejero et al. [27] demonstrate that LLMs outperform conventional machine learning techniques for the task of attack classification in Internet of Things (IoT) logs. Moreover, they map attack classes to entries of the CAPEC[5] attack pattern database and thereby show that LLMs are able to enrich detections with mitigation recommendations tailored to IoT environments.

The interpretive capabilities of LLMs are also suitable to explain intrusion detection alerts. Balogh et al. [14] employ LLMs to analyze alerts generated by a network intrusion detection system and map them to MITRE ATT&CK techniques. Although some correct classifications are achieved, the authors conclude that the models are currently insufficiently reliable and too resource-intensive for automated deployment in practice. In contrast, Hans et al. [8] report high classification accuracy achieved on similar data sets. In addition to mapping alerts to tactics and techniques, they investigate alert grouping, i.e., segmenting continuous alert streams into clusters representing individual attacker actions or attack steps. Zha et al. [28] also address the alert grouping problem by using LLMs to infer the root causes of alerts, specifically to trace cascading effects of service failures that often cause alert floods.

2.2. Data Sets

Publicly available log data sets play a central role in the development and evaluation of methods for log analytics, as they enable reproducible experimentation and fair comparison across approaches. Historically, however,

[5] `https://capec.mitre.org/`

Table 1: Overview of reviewed data sets.

Dataset	Year	Labels	Generation	Number of Tactics	Number of Techniques	System Logs	Network Traffic	HIDS Alerts	NIDS Alerts	Environment	Network
Supercomputer Logs [29]	2006	Failure Type	Real	0	0	✓				Supercomputer	Generic
CICIDS2017 [17]	2017	Attack Type	Scripted	10*	19*	✓	✓			Windows + Linux	Enterprise
Loghub [30]	2019	Various	Mixed	0	0	✓				Various	Various
Mordor/OTRF [20]	2019	Technique	Scripted	9	35	✓	✓			Windows + Linux	Enterprise
AIT-LDSv1.1 [31]	2020	Attack Type	Scripted	7*	15*	✓				Linux	Enterprise
CYSAS-S3 [32]	2020	Attack Type	Scripted	8	11	✓	✓	✓	✓	Windows + Linux	Military
DAPT-2020 [33]	2020	Attack Type	Red Team	4	16	✓	✓		✓	Linux	Enterprise
EVTX-ATTACK-SAMPLES [21]	2020	Technique	Scripted	11	58	✓				Windows	Enterprise
PWNJUTSU [34]	2022	None	Red Team	5	13	✓	✓			Windows + Linux	Generic
UWF-ZeekData22 [18]	2022	Technique	Red Team	14	36		✓			Windows + Linux	Generic
AIT-LDSv2.0 + AIT-ADS [25, 35]	2022	Attack Type	Scripted	8	10	✓	✓	✓	✓	Linux	Enterprise
Unraveled [36]	2023	Technique	Manual	5	12	✓	✓		✓	Windows + Linux	Enterprise
PicoDomain [37]	2023	Attack Type	Scripted	8*	19*		✓			Windows	Enterprise
LMDG [38]	2024	Attack Type	Scripted	5*	10*	✓	✓			Windows	Enterprise
UWF-ZeekData24 [19]	2024	Technique	Scripted	7	5		✓			Windows + Linux	Enterprise
APT-Persistence [39]	2024	None	Scripted	6*	11*	✓		✓		Windows	Enterprise
ATLASv2 [40]	2024	Attack Type	Manual	13*	30*	✓		✓		Windows	Generic
EVTX-to-MITRE-Attack [22]	2024	Technique	Scripted	11	49	✓		✓		Windows	Enterprise
Linux-APT [41]	2024	Technique	Manual	12	26	✓		✓		Linux	Generic
Atomic-EVTX [23]	2025	Technique	Scripted	13	121	✓				Windows	Generic
CasinoLimit [42]	2025	Technique	Red Team	14	67	✓	✓		✓	Linux	Enterprise
COMISET [24]	2025	Technique	Red Team	11	50	✓				Windows	Enterprise
CIDDS-113 [43]	2026	Attack Type	Scripted	5*	6*	✓	✓	✓		Windows + Linux	Enterprise
CAM-LDS (Our Dataset)	2026	Technique	Scripted	13	81	✓	✓	✓	✓	Linux	Enterprise

** Asterisks indicate that tactics and technique counts are not explicitly stated in the paper and thus estimated based on attack descriptions.*

high-quality data sets have been sparse, particular those that fulfill common requirements such as correctness, relevance, realism, and timeliness [25, 44]. In recent years, several log data sets have emerged as de facto benchmarks, and a number of newly published data sets constitute suitable candidates for specific research directions, including LLM-based log interpretation. In the following, we review existing system log data sets, compare their key characteristics, and highlight notable properties of individual contributions.

Table 1 summarizes relevant properties of all reviewed data sets, including the types of collected data, the approach to attack generation, the granularity of labels, the operating systems from which logs are captured, the emulated network topology, and the number of covered attack tactics and techniques corresponding to the MITRE ATT&CK framework. For comparability, we only count techniques, because not all data sets provide labels at the sub-technique level; where necessary, sub-techniques are therefore mapped to their corresponding techniques.

Some of the most widely used log data sets, in particular in the research area of anomaly detection, are provided in the LogHub repository [30] as well as the collection of supercomputer logs originally described by Oliner et al. [29]. These log data sets primarily comprise logs from normal system behavior as well as events that indicate failures; however, there are no traces of cyber attacks present in the data. Consequently, their relevance to cyber-security–focused research is limited. Nonetheless, they are suitable for evaluation of generic approaches that leverage LLMs for anomaly detection or log interpretation as presented in Sect. 2.1.1.

Several data sets have been collected specifically for evaluating intrusion detection systems, with a primary focus on network traffic [45]. For example, the authors of PicoDomain [37] simulate an enterprise environment and prepare a kill chain comprising client-side exploitation, privilege escalation, and data exfiltration. Data collection relies on the network monitoring tool Zeek[6], which produces network-centric log files. The CICIDS2017 [17] data set includes both network traffic and system log data; however, since only network traffic is labeled, it is predominantly used for network intrusion detection. A similar limitation applies to DAPT-2020 [33], which collects system logs from various sources such as syslog and audit, but only provides labels for network traffic.

[6] `https://zeek.org/`

The UWF-ZeekData22 [18] data set was collected during a cyber security exercise in which students employed both defensive and offensive tools. While the labeled data set comprises attacks from all 14 MITRE ATT&CK tactics, the vast majority of events corresponds to reconnaissance activities such as port scanning, and data collection is limited to Zeek logs. UWF-ZeekData24 [19] is a follow-up data set to UWF-ZeekData22 that replaces student-driven attacks with scripted executions, which reduces timestamp inaccuracies, labeling errors, and noise according to the authors.

Beyond network-centric data sets, several data sets that include system logs rely on red teaming exercises for attack execution. The PWNJUTSU data set [34] contains attack traces generated by 22 offensive security experts and includes both network traffic and diverse system logs, such as Linux authentication, access, and audit logs, as well as Windows event logs. The authors of CasinoLimit [42] emphasize that capturing both network traffic and system logs is important to reconstruct attack scenarios. They emulate the network infrastructure of a casino and task 114 penetration testers with attacking the environment. Subsequently, they manually label the resulting data set with 67 distinct attack techniques observed over a time span of 10 hours. The authors of COMISET [24] rely on rule-based labeling of attack traces produced by a red team targeting an emulated small enterprise network of Windows hosts, yielding 50 techniques across 11 tactics in two environments.

While red-teaming and penetration-testing campaigns typically provide broad coverage of attack behaviors [19], data sets that focus on specific tactics and settings often rely on scripted attacker behavior. For example, the APT-Persistence data set [39] concentrates on persistence techniques, LMDG [38] focuses on lateral movement, CIDDS-113 [43] centers on file-encryption attacks, and CYSAS-S3 [32] targets a military-oriented use case.

Most existing data sets, however, model attack scenarios that follow a full or partial kill chain and therefore span multiple tactics. AIT-LDSv1.1 [31] includes network scanning, account discovery, brute-force initial access, exploitation for persistence, and privilege escalation, and was designed to support the evaluation of host-based intrusion detection systems. A distinguishing feature of this data set is the collection of fine-grained Linux audit logs. The same authors introduce AIT-LDSv2.0, which includes additional attack vectors, collects logs from multiple hosts, and improves event labeling. In another follow-up publication, the authors present AIT-ADS [35], an alert data set generated by forensically processing the AIT-LDSv2.0 with

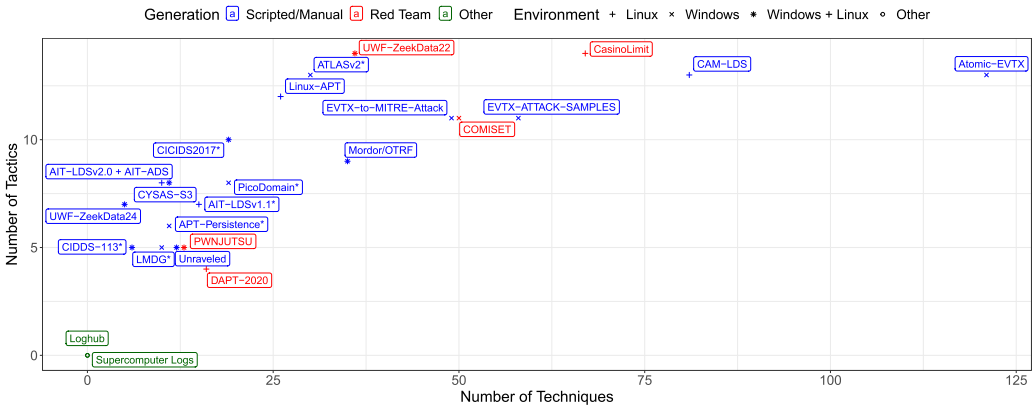


Figure 1: Number of MITRE ATT&CK tactics and techniques covered by the reviewed data sets. Asterisks indicate that tactics and technique counts are not explicitly stated in the paper and thus estimated based on attack descriptions.

signature- and anomaly-based intrusion detection systems. The attack chain emulated in the ATLASv2 [40] data set relies on phishing for initial access and aims at the exfiltration of documents. Captured data sources involve audit logs, browser application logs, DNS logs, and security alerts from a host-based intrusion detection system. Notably, the authors emphasize manual execution of the attack steps to enhance realism.

While the aforementioned data sets employ descriptive attack type labels, several other data sets adopt tactic- and technique-level labeling aligned with the the MITRE ATT&CK framework. This includes several publicly available data sets not directly associated with research publications, such as Atomic-EVTX [23], EVTX-to-MITRE-Attack [22], and EVTX-ATTACK-SAMPLES [21]. While these data sets cover a large number of techniques, they are restricted to Windows environments. The Mordor/OTRF [20] data set extends coverage to Linux hosts and web services in addition to Windows event logs. Unraveled [36] builds upon DAPT-2020 [33] by expanding attack tactics, improving IDS deployment, and refining labels. Linux-APT [41] simulates the behavior of selected APT groups and provides technique-level labels of attacks against Linux-based environments.

In summary, several existing data sets are ill-suited for log interpretation in a cyber-security context, either because they do not contain attack traces or because they focus primarily on network traffic rather than system logs. As illustrated in Figure 1, many data sets cover fewer than nine tactics

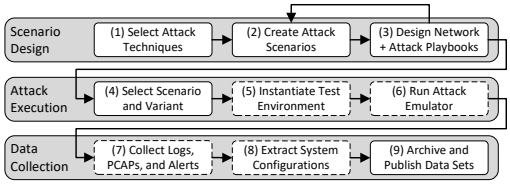


Figure 2: Overview of our methodology. Dashed boxes indicate automated steps that require minimal manual intervention.

and fewer than twenty techniques, which constrains the scope of evaluations conducted on them. Data sets with broader coverage of attack techniques typically originate from red teaming exercises, which introducing challenges related to controllability, reproducibility, and labeling reliability [19], or are limited to Windows-only environments, thereby excluding Linux-specific log sources.

To address these limitations, we introduce CAM-LDS, a system log data set collected through seven fully scripted attack scenarios that cover 81 distinct attack techniques executed in a reproducible Linux environment. Accordingly, we consider it as a complementary data set to existing Windows-centric data collections. In addition to system logs and network traffic, CAM-LDS includes alerts generated by host- and network-based intrusion detection systems, as well as system configuration data that provides valuable contextual information for log interpretation and attack analysis.

3. Data Set Generation

This section describes our approach to generate the CAM-LDS. After explaining our methodology, we present the network topology common to all scenarios, the adversarial emulation tool used to execute the attacks, and the corresponding attack chains of each scenario.

3.1. Methodology

In this section, we describe our methodology for generating log data sets. We thereby refer to the Steps (1)-(9) displayed in Fig. 2, which summarizes our methodology.

3.1.1. Scenario Design

Our methodology is driven by the objective of collecting log data corresponding to a wide variety of attack techniques. Accordingly, Step (1) of

our methodology focuses on the systematic selection of suitable techniques from the MITRE ATT&CK framework, one of the most widely adopted and up-to-date references for modeling adversary behavior. At the time of writing, the ATT&CK Enterprise matrix comprises 216 distinct techniques[7]. We review each technique individually and assess it according to the following criteria: (i) *Observability*. We base our selection on the "Detection Strategies" specified for each technique in the MITRE ATT&CK framework. These recommendations describe how adversary behavior can be detected in practice and often indicate relevant data sources. We therefore focus on techniques whose detection strategies suggest the generation or modification of observable artifacts in system or application logs. Techniques that do not produce such traces are of limited relevance for log-centric analysis and are consequently excluded. For example, we exclude most techniques associated with the "Resource Development" tactic, which primarily describes activities that occur outside of the target infrastructure, such as acquiring domains or social media accounts. (ii) *Feasibility.* We restrict our selection to techniques that can be realistically emulated within our Linux-based test environment. As a result, techniques that are specific to other platforms, such as the Windows operating system, various mobile systems, or cloud-native environments, are excluded. (iii) *Automation*. We require that execution of the attack technique can be fully automated to support scripted attack chains and ensure reproducibility. Consequently, techniques that inherently involve manual interaction with physical hardware, such as inserting USB devices, are omitted.

After applying these criteria to the entire framework, we end up with a set of 122 attack techniques for potential implementation. Thereby, in order to construct a challenging data set where some individual attack steps are only recognizable as such when considered in the context of the entire kill chain, we purposefully keep techniques that could yield manifestations similar to those of benign system behavior. In particular, this includes activities commonly performed by administrators, such as system information collection.

Step (2) aims to create attack scenarios based on the initial set of 122 attack techniques. We start by grouping techniques according to their infrastructure requirements, such as the need for specific services or particular network configurations. Subsequently, we arrange the grouped techniques

[7]This work relies on version 18.1 of the MITRE ATT&CK Enterprise matrix, which is the most recent version available at the time of writing.

into coherent and realistic attack sequences, following common kill chain models and reflecting the complexity of real-world attacks in terms of technique diversity [46], ultimately resulting in seven attack scenarios. At this stage, we exclude some techniques that cannot be meaningfully integrated into any scenario. We further observe that several techniques are functionally interchangeable, and therefore introduce variants in our scenarios, where individual attack steps are realized using different techniques while achieving the same attacker objective. After completing this step, we obtain a conceptual specification of both the required network infrastructure and the complete set of attack scenarios.

The technical realization of the infrastructure and the implementation of the attack emulation playbooks in Step (3) is carried out as an iterative task that is coupled with Step (2). The reason for this is that during development and testing, techniques may be moved between or duplicated to other scenarios to produce more informative or diverse log manifestations. Moreover, certain techniques that turn out impractical or unsuitable for the environment or attack chains are removed, and the scenarios are revised accordingly. Note that during this phase we also carry out validation activities to ensure that all scenarios are logically consistent, correctly implemented, and accurately labeled with the corresponding attack techniques. Thereby, all attack playbooks and scripts are independently cross-checked by two other experts. After completion of the implementation phase, we eventually end up with a fully scripted infrastructure as well as playbooks for 81 attack techniques across seven scenarios.

3.1.2. Attack Execution

In Step (4), we select one of the previously designed attack scenarios, which are accompanied by configuration files that specify the required network topology as well as the necessary software components and versions. Note that the subsequent steps refer to a single simulation run; however, we repeat these steps for every scenario and for all possible combinations of scenario variants. Step (5) instantiates the infrastructure for the selected scenario in a virtualized environment, which is done from scratch for every scenario to avoid that any traces left from previous attacks remain on the systems and to ensure that the infrastructure is in a predictable state. Deployment of the network and all hosts is fully automated, as the entire in-

frastructure is specified using Infrastructure-as-Code. We employ OpenTofu[8] and Terragrunt[9] to provision the network components and virtual machines, and use the configuration management framework Ansible[10] to install and configure all software components on the deployed hosts.

Step (6) executes the attack emulation for the selected scenario and, where applicable, its variants. Note that we do not start the emulation tool immediately after infrastructure deployment, as newly instantiated systems may generate background log events during and shortly after startup. To prevent such events from overlapping with attack traces, we introduce a stabilization period of 15 minutes before starting the attack emulation. In addition, log events may be generated with slight delays following individual attack actions, for instance, due to communication overheads or buffering when large numbers of events are generated. To account for this, we enforce a minimum sleep interval of 15 seconds between consecutive attack steps.

3.1.3. Data Collection

After completion of the attack execution, Step (7) collects log data from 18 distinct data sources and across all hosts in the environment. This includes standard Linux log sources such as audit logs, system logs (syslog), authentication logs, access and error logs, package management logs, and CRON logs. In addition, we collect application-specific logs from services deployed in the scenarios, including Zoneminder, FTP, Puppet, Docker, Nextcloud, and the mail server, as well as system performance metrics. We further collect alerts generated by the host-based intrusion detection system Wazuh[11], which we utilize with its default rule set. Netflows and network packet captures are captured using Suricata[12], which simultaneously serves as our network-based intrusion detection system and is also used with standard detection rules.

Step (8) extracts system configurations from all hosts. These configurations provide important contextual information for forensic analysis, such as determining the presence of vulnerable software versions or insecure system settings. In the final Step (9), we archive all collected data and release them in their raw format without modification to ensure that no potentially rele-

[8] `https://opentofu.org/`
[9] `https://terragrunt.gruntwork.io/`
[10] `http://www.ansible.com/`
[11] `https://wazuh.com/`
[12] `https://suricata.io/`

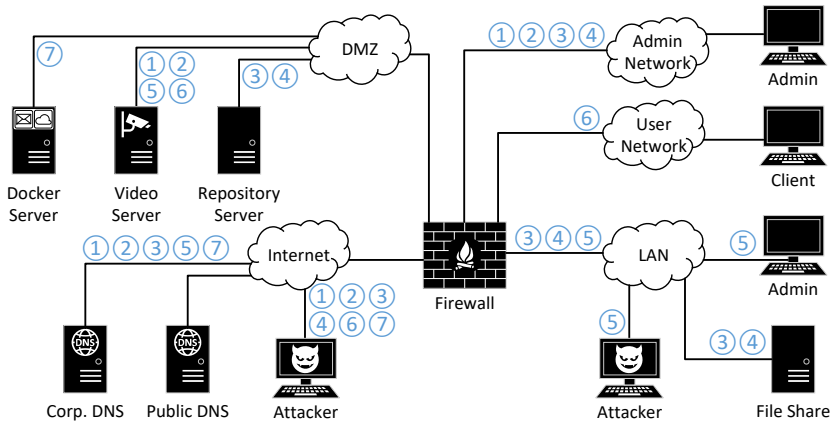


Figure 3: General network topology and scenario-specific deployments of components.

vant information is lost. Anonymization is not required, as no human users interact with the infrastructure during the simulations.

3.2. Network Topology

Cyber attacks rarely affect a single host in isolation; instead, their traces typically manifest across multiple interconnected systems. Gaining a comprehensive understanding of attacker behavior, e.g., as part of forensic investigations conducted after an incident, therefore requires correlating log data from multiple hosts and infrastructure components. To capture attack traces from a realistic setup of such a distributed systems, we design a network topology that approximates the IT infrastructure of a small enterprise as illustrated in Fig. 3.

Following established best practices for network segmentation [47], the topology is divided into five security zones that are interconnected via a firewall: (i) *Internet*. This zone provides external connectivity and contains a public DNS server as well as a corporate DNS server for the simulated organization. In all but one of our scenarios, the attacker gains initial access through the Internet zone. (ii) *Demilitarized Zone (DMZ)*. The DMZ hosts several publicly exposed services, including a Docker server running a mail platform and a cloud service as containers, a repository server used for internal asset management, and a video server providing video surveillance functionality. (iii) *Local Area Network (LAN)*. The LAN provides internal services to the organization, including a file share accessible to employees. (iv) *User Network*. This zone represents the standard working environment

of employees, who access the enterprise network using their company workstations. (v) *Admin Network*. The admin network is reserved for network administrators and serves as the default zone for privileged access to the infrastructure.

The network is instantiated on a per-scenario basis, deploying only those components that are required for executing the attack and for collecting the corresponding traces. Figure 3 shows which components are deployed in each scenario by referencing the number of the respective scenario in blue circles. For example, in Scenario 5 we do not deploy the file share, but assume that a network administrator is connected to the LAN. In this scenario, the attacker also accesses the network through the LAN rather than the Internet. The next section describes our method for emulating the attacker's behavior in detail.

3.3. Attack Emulation

Our attack emulation strategy is guided by two primary requirements: repeatability and realism of the generated log artifacts. Repeatability is essential to enable multiple executions of identical attack scenarios, which in turn allows to verify the consistent generation of attack manifestations across runs. Moreover, repeatability enables automation of attack execution, thereby reducing manual effort during data collection and minimizing inconsistencies as well as human error. Furthermore, automated and repeatable attack scenarios facilitate the systematic generation of variants in which individual steps of an attack chain are replaced or modified, allowing us to cover additional attack techniques in a controlled manner. Since neither red teaming exercises nor manual attack execution (cf. Sect. 2.2) enable repeatable and automated experiments that yield consistent and reproducible results [19], we opt for script-based attack execution.

Naive automation through scripts can negatively affect the realism of observed attack traces [40, 48]. Specifically, it is non-trivial to realize system interactions with simple shell scripts in such a way that they are indistinguishable from actual attacker behavior. For example, human users generally rely on interactive programs such as text editors to modify files, while scripts are limited to non-interactive mechanisms such as file replacement and stream editing to achieve equivalent actions. These differences are visible in system logs and may lead to artifacts that are uncharacteristic of real-world attacks, thereby reducing the authenticity of the resulting data sets and possibly affecting the validity of any evaluation results.

```
vars:
  $MAIL_HOST: mail.attackbed.com
  $DOMAIN: attackbed.com
  $DNS_LIST: /usr/local/share/SecLists/Discovery/DNS/subdomains-top1million
      -5000.txt

commands:
- type: shell
  cmd: dnsenum -f $DNS_LIST --dnsserver 192.42.0.233 $DOMAIN
  metadata:
    techniques: "T1590.002, T1591"
    description: "Enumerate subdomains of corporate domain-zone"

- type: shell
  cmd: ./smtp-user-enum.pl -M VRFY -U smtp-user.txt -t $MAIL_HOST
  metadata:
    techniques: "T1589.002"
    description: "Enumerate email addresses on SMTP-Host"

- type: shell
  cmd: hydra -l "alice@$DOMAIN" -P smtp-pass.txt -c 5 -s 143 $MAIL_HOST imap
  metadata:
    techniques: "T1078.002, T1110.001, T1133"
    description: "Brute-force IMAP-password using the already enumerated
        username"
```

Figure 4: Sample AttackMate playbook for DNS enumeration, user identification, and brute-force login.

To reconcile the requirements of repeatability and realism, we employ AttackMate [48], an open-source adversarial emulation framework designed specifically for reproducible experiments with realistic attack execution across all phases of the kill chain. AttackMate uses fully scripted playbooks to define attack chains in a deterministic and repeatable manner. Figure 4 shows an example playbook consisting of three sequential attack steps. Each step executes a predefined command on the target system, yielding consistent log traces as long as the underlying infrastructure remains unchanged. During execution, AttackMate generates a timestamped output file that associates each command with the corresponding attack labels specified in the metadata fields. These time-based labels enable precise correlation between attack steps and collected log data or network traffic.

At the same time, AttackMate preserves realism by natively supporting interactive sessions and sending individual keystrokes that are indistinguishable from manual typing. Unlike many existing adversarial emulation tools that rely on agents installed on target hosts, AttackMate operates exclusively from a dedicated attacker machine and interacts with the environment solely through remote interfaces such as SSH, thereby accurately modeling an external adversary. In addition to shell command executions, AttackMate further provides dedicated modules for common attack vectors and interfaces

with offensive tools widely used by adversaries, such as Metasploit[13].

3.4. Scenarios

Based on the infrastructure setup described in Sect. 3.2, we implement seven attack scenarios as AttackMate playbooks. The scenarios and their respective steps have been designed to represent realistic kill chains that cover many distinct techniques from the MITRE ATT&CK framework. Table 2 provides a concise overview of our seven scenarios, including their title, a description summarizing the overall setup and the main attack vectors, the number of variants, the resulting simulated attack paths, and the number of labeled attack steps. In total, our data set comprises 34 simulations and 243 attack steps across all scenarios.

Subsequent sections describe each scenario in detail and include visualizations of the corresponding attack paths. In these state chart diagrams, each block represents one or more attack steps associated with a specific set of technique labels; individual steps correspond to single attacker actions and are identified by the numbers shown within the block. Note that even though execution of some attack techniques only requires a single step, several techniques are realized through sequences of steps and thus grouped for brevity. Furthermore, we indicate the tactics corresponding to the involved techniques using markers above each block, namely Reconnaissance (RCN), Resource Development (RSD), Initial Access (IA), Execution (EXE), Persistence (PST), Privilege Escalation (PVE), Defense Evasion (DEF), Credential Access (CRD), Discovery (DSC), Lateral Movement (LAT), Collection (COL), Command and Control (CNC), Exfiltration (EXF), and Impact (IMP).

3.4.1. Scenario 1: Video Server Exploit

As depicted in Fig. 5, Scenario 1 starts with reconnaissance, which is the earliest stage of the kill chain. We do not assume that the attacker has any prior knowledge about the target network beyond its public domain *attackbed.com*. The attacker's first objective is information gathering, which we subdivide into several steps. In the first step, the attacker enumerates subdomains of the corporate DNS server using *dnsenum*[14], which reveals *video.attackbed.com*, *fw.attackbed.com*, and *repo.attackbed.com*. The attacker

[13] `https://www.metasploit.com/`

[14] `https://github.com/fwaeytens/dnsenum`

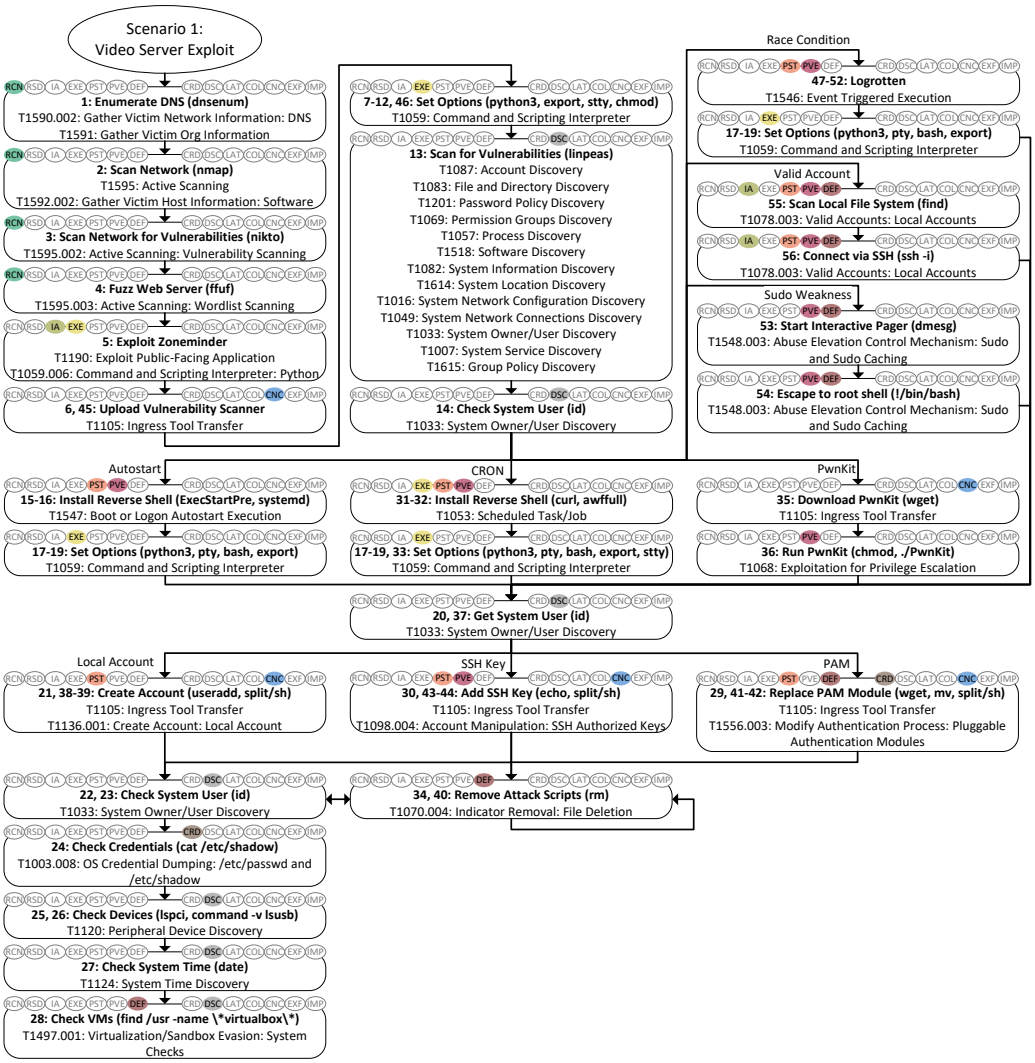


Figure 5: Attack paths of Scenario 1: Video Server Exploit.

Table 2: Overview of attack scenarios.

ID	Name	Description	#Variants	#Sim.	#Steps
1	Video Server Exploit	In addition to some reconnaissance and scanning attacks, this scenario revolves around compromising the video server. A key element in this attack chain is the exploitation of the vulnerable video server software. Subsequently, the attack graph involves five variants that allow the attacker to escalate to root privileges as well as three variants that provide the attacker with persistent access; we simulate all combinations of these variants.	6 (PVE) 3 (PST)	18	56
2	Linux Malware	The malware considered in this scenario are an implant that establishes reverse-shell connection as well as a rootkit that targets dynamic linking. The remainder of the scenario broadly covers discovery techniques and exfiltrates some data from the compromised machine.	2 (IA)	2	29
3	Lateral Movement	After logging into the firewall through brute-force password guessing, the attacker moves laterally to the file share in the LAN segment. This is achieved by injecting malicious code either into a shared service file, a software management tool, or a manipulated package. The scenario ends with several destructive activities on the target host, including data encryption and deletion. The entire scenario is implemented for both SSH and VNC connections.	2 (IA) 3 (LAT)	6	60
4	Network Attack	This scenario aims to compromise the firewall of the network. A port knocking sequence is used to download and run implants, which in turn enable the attacker to modify firewall rules and reach even more hosts in the network.	-	1	22
5	Network Sniffing	In this short scenario, the attacker captures authentication tokens from network traffic and re-uses them to gain access to a system.	-	1	3
6	Attack on Client	This scenario considers attack cases where a human user is deceived into compromising their own computer, either by downloading and opening a malicious document, installing a fake browser plugin, or providing the attacker access via screen sharing and remote control software. Once inside the system, the attacker installs a backdoor for permanent access. The scenario also involves several activities for data collection, including a keylogger, a data exfiltration tool, and extraction of clipboard data.	3 (IA) 2 (PST)	5	60
7	Docker Attack	In this scenario, the attacker first carries out some scanning activities and then brute-forces a password of a mail service. By re-using the collected credentials on a cloud storage service, the attacker is able to execute an authenticated exploit that grants them access to the containerized environment that hosts both the mail and cloud storage services. Once inside, the attacker exploits weak configurations to escape the container and obtain root access on the host.	-	1	13

then carries out network mapping of the video subdomain with *nmap*[15], which is useful to discover available services on hosts, and identifies that port 80 (HTTP) is open on the video server. Next, a network vulnerability scan with *nikto*[16] shows that the server runs Apache and that the main application entry point is *video.attackbed.com/zm/*, which suggests the presence of the open-source video surveillance software ZoneMinder[17]. The attacker then performs deeper web content discovery through fuzzing with *ffuz*[18], a fast

[15] `https://nmap.org/`
[16] `https://cirt.net/nikto/`
[17] `https://zoneminder.com/`
[18] `https://github.com/ffuf/ffuf`

web fuzzer that enumerates paths using a wordlist.

Since fuzzing does not yield any direct ways to compromise the system, the attacker searches public databases for recent ZoneMinder vulnerabilities and identifies CVE-2023-26035[19], which describes a vulnerability in ZoneMinder's snapshot action that enables unauthenticated remote code execution. The attacker then resorts to the Metasploit framework, where they find and execute an already implemented exploit for that vulnerability.

With the ability to execute arbitrary commands on the video server, the attacker configures an interactive shell and pursues their new objective of escalating their privileges. To enable fast enumeration of potential privilege escalation vectors, the attacker uploads and executes *linpeas*[20], a vulnerability scanner for the Linux operating system that reports misconfigurations, weak permissions, and other common escalation paths as indicated by the broad coverage of attack techniques in Fig. 5. After executing the *id* command to record current user context, the attacker proceeds to escalate their privileges through one of five possible variants modeled in Scenario 1, some of them based on the linpeas output. (i) *Autostart.* The attacker uploads a reverse-shell, opens the ZoneMinder systemd unit file with an interactive text editor, and adds a line that executes the payload on service start, e.g., when the system is booted. When the unit is started, the payload is executed and the attacker gains root access through the reverse-shell. (ii) *CRON.* In this variant the attacker injects commands that download and run a reverse-shell into a system utility that is executed periodically by CRON. For the purpose of Scenario 1, we assume that the *awffull*[21] tool, a log analysis script that runs under a privileged account and is managed by CRON, was misconfigured by system operators during installation. In particular, vulnerable permissions of the corresponding CRON-job allow unprivileged users to inject commands that are subsequently executed with root permissions. Accordingly, as soon as CRON invokes the manipulated script, the injected commands establish the reverse-shell and provide the attacker with root access to the system. (iii) *PwnKit.* The attacker downloads and runs the *PwnKit*[22] script, which is a self-contained exploit for *polkit's pkexec utility*[23] that provides root access.

[19] `https://nvd.nist.gov/vuln/detail/CVE-2023-26035`
[20] `https://github.com/peass-ng/PEASS-ng/tree/master/linPEAS`
[21] `https://packages.debian.org/sid/web/awffull`
[22] `https://github.com/ly4k/PwnKit`
[23] `https://nvd.nist.gov/vuln/detail/cve-2021-4034`

(iv) *Race Condition.* Most Linux distributions come by default with *logrotate*[24], a service that enables automatic rotation, compression, and retention of log files. Unfortunately, this tool is vulnerable to a race condition when it runs under root privileges but the managed log files are located in directories accessible to unprivileged users. In this variant, the attacker makes use of the *logrotten*[25] exploit, which replaces one of the log files with a symbolic link to an attacker-controlled directory just at the right time so that logrotate creates the new log file in the linked directory. Since the attacker has write access to the file managed by a root process, they can use it to inject a reverse-shell. (v) *Sudo Weakness.* This variant assumes that due to a system misconfiguration, non-privileged users are able to execute *dmesg*, which is a utility to print the kernel ring buffer. By running *dmesg* interactively and abusing its shell-escape features, the attacker is able to spawn a root shell. (vi) *Valid Account.* In this simple variant, the attacker scans the local file system and thereby recovers a private SSH key belonging to a high-privileged user account, which they use to establish a privileged SSH session.

After completing each variant for privilege escalation, the attacker runs the *id* command to confirm their newly obtained privileges. At this stage of the attack chain, the attacker's objective is to obtain persistence to retain access to the system even if the previously exploited vulnerabilities are patched, e.g., when system operators upgrade ZoneMinder to a more recent version that is not vulnerable to the exploit. Scenario 1 involves three variants for persistence that we describe in the following. (i) *Local Account.* The attacker creates a new local user with the name *webadmin* to not trigger any suspicions. Then, the attacker transfers their own SSH key to the authorized keys. (ii) *SSH Key.* This variant is similar to the previous one, with the exception that the attacker copies their own SSH key into the authorized keys of an already existing privileged user. (iii) *PAM.* The attacker replaces the system's *Pluggable Authentication Module* (PAM) with a modified variant that provides backdoor access.

Note that for some of these variants, the attacker makes use of the split command to obfuscate their activities and reduce traceability. However, since execution of the split command depends on the type of root shell, we are only able to model this optional technique as part of the *PwnKit*, *Sudo Weakness*,

[24] `https://linux.die.net/man/8/logrotate`
[25] `https://github.com/whotwagner/logrotten`

and *Valid Account* variants for privilege escalation. All other variants execute the aforementioned commands directly.

After establishing persistence, the attacker runs the id command once more and removes some of the files downloaded as part of the previous attack steps. The attacker then performs targeted post-compromise discovery, in particular, extraction of credentials and hashed passwords for offline cracking, enumeration of connected peripheral devices, checking of system time and timezone, and searching for virtual machines or hypervisor artifacts.

3.4.2. Scenario 2: Linux Malware

Figure 6 visualizes the attack chain for Scenario 2. We assume that the attacker has already compromised the video server and obtained root privileges, e.g., through an attack chain comparable to Scenario 1. The attacker's immediate objective in Scenario 2 is to compromise the system further through the installation of malware, which we model in two variants. (i) *CRON*. The attacker connects to the target machine via SSH, starts a HTTPS listener, and transfers an implant via SFTP. The implant is installed by modifying the *awffull* script that has already been targeted in the *CRON* variant of Scenario 1; however, instead of editing the file with a text editor, the attacker uses a stream editor to inject a command that starts the implant. (ii) *Rootkit*. The attacker again connects via SSH, prepares a rootkit, and transfers it via SFTP. Analogous to the previous variant, the attacker also transfers an implant and starts a HTTPS listener. The implant is installed by editing ZoneMinder's systemd unit file similar to the *Autostart* variant of Scenario 1. The rootkit is installed through a dynamic-linker preload technique so that it is loaded system-wide into dynamically linked processes [49]. Finally, the attacker reloads systemd and restarts ZoneMinder's service to trigger execution of the implant through systemd.

After completion of either variant, the attacker's subsequent objective is to collect and exfiltrate relevant information. The post-exploitation phase begins with discovery operations: enumeration of files, running processes, and active network connections, as well as collection of network interface configuration. The attacker then acquires artifacts for offline analysis, including process memory snapshots and a process identifier for the database service. Subsequently, the attacker extracts stored authentication material such as hashed credentials. Collected data is aggregated into archives and extracted; the attacker also removes local artifacts that would reveal the exfiltration event. In addition to bulk collection, the attacker selectively downloads in-

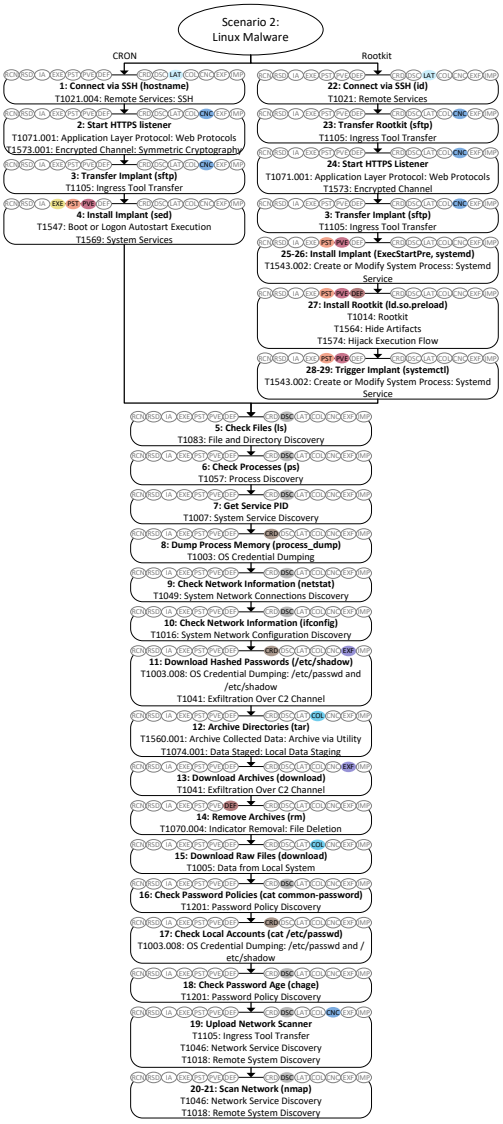


Figure 6: Attack paths of Scenario 2: Linux Malware.

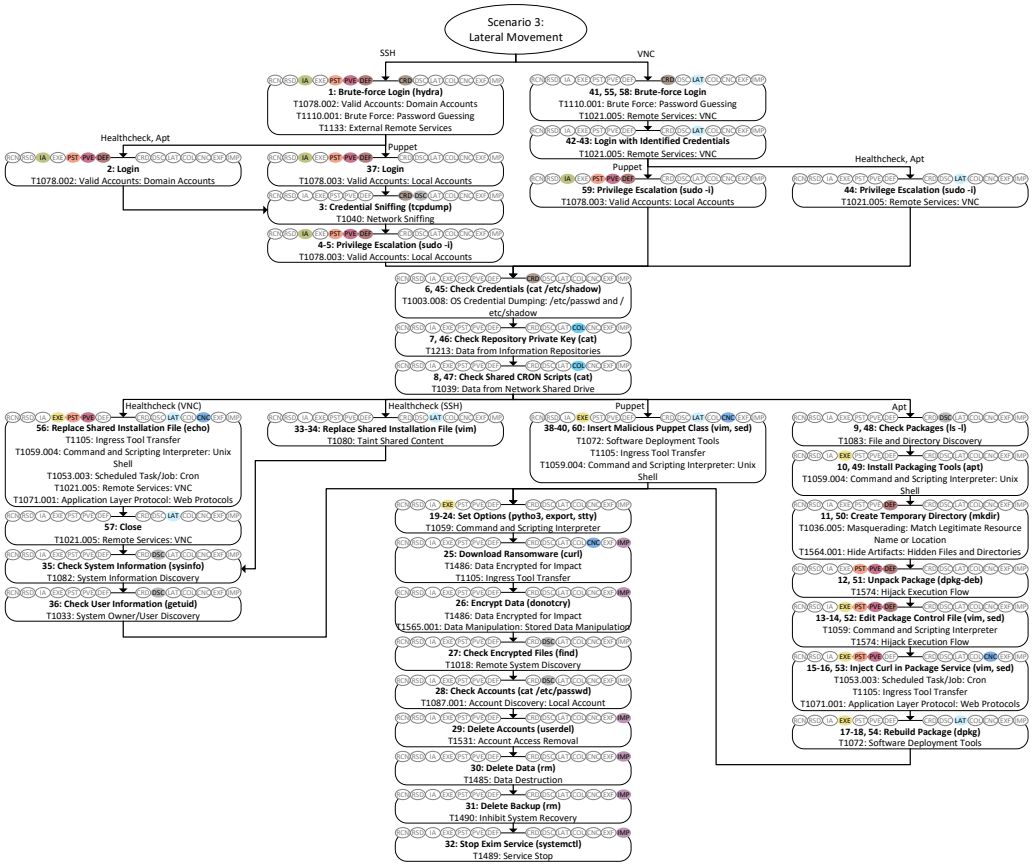


Figure 7: Attack paths of Scenario 3 Lateral Movement.

dividual files and directories of interest. To support further lateral discovery, the attacker deploys *nmap* that was already used in Scenario 1 and runs it from the compromised host to enumerate reachable hosts and services.

3.4.3. Scenario 3: Lateral Movement

Scenario 3 emulates how an attacker moves within the network from one host to another, in particular from the initially compromised puppet server to the file share within the LAN segment. Figure 7 visually summarizes the entire scenario. The scenario involves two variants regarding the way how the attacker connects to the targeted network. (i) *SSH*. The attacker attempts to log into the puppet server via SSH by brute-forcing several combinations of user names and common passwords. To this end, the attacker employs

hydra[26], a tool that automatically and rapidly goes through a pre-defined file of login credentials and tests whether some of them provide access. For the purpose of our scenario, we assume that one of the users with root privileges has a weak password contained in the list. After logging into the Puppet server, the attacker deploys the network sniffer *tcpdump* to capture credentials transmitted over the network. In our scenario, a scheduled CRON job on the file share periodically retrieves data from the Puppet server via FTP. The attacker intercepts the credentials contained in one of these connections and subsequently re-uses them to escalate to root privileges. (ii) *VNC*. In this variant, the attacker attempts to brute-force a connection via Virtual Network Computing (VNC) rather than SSH. Again, we assume that the attacker manages to guess a weak password of a privileged user after a few attempts, allowing them to log into the puppet server.

Both of the aforementioned variants continue with the attacker printing hashed passwords for offline cracking. Subsequently, they check for the presence of private keys and find one corresponding to the puppet server. Then, the attacker locates a CRON job called *healthcheck* in a Network File System (NSF) share. At this point, we consider several variants how the attacker exploits shared system content to move laterally to another server in the network. Note that while conceptually identical, the technical implementations of these steps are realized differently depending on whether the attacker accesses the system through SSH or VNC as determined by the first variants of the scenario. (i) *Healthcheck*. The attacker edits the *healthcheck* CRON job and injects a line that downloads and establishes a reverse-shell. Once this script is automatically executed on the file share as part of normal *healthcheck* activity, the attacker gains access on that system and has thus achieved lateral movement. (ii) *Puppet*. The attacker realizes that Puppet[27], a service that automates administrative tasks such as server configuration in networks, is installed on the firewall and thus likely used across multiple nodes in the network. The attacker then edits parts of puppet's code; in particular, they inject a malicious class that downloads and executes a reverse-shell and assign that class to the node corresponding to the file share. Next time the Puppet service running on the file share executes that code, the reverse-shell triggers and provides the attacker with

[26] `https://github.com/vanhauser-thc/thc-hydra`
[27] `https://github.com/puppetlabs/puppet`

access to that system. (iii) *Apt*. The attacker realizes that the puppet server provides APT packages to other servers, including the file share that updates the *healthcheck* package through puppet. They then decide to build a malicious version of that package that appears as an update. This is achieved by downloading packaging tools, unpacking the *healthcheck* package, adding a malicious CRON job that establishes a reverse-shell to the service, and rebuilding the package. As soon as the admin user triggers the installation of the package, e.g., as part of normal updating procedures, the injected code establishes a reverse-shell and thus allows the attacker to move laterally to the file share. Note that we also emulate administrator activities analogous to attacker behavior to ensure that these attack steps succeed.

After completing any of the aforementioned variants, the attacker first configures the shell for interactive access and then continues with a sequence of destructive attack techniques. First, the attacker downloads a ransomware from their own system and uses it to encrypt the media directory on the system. After checking that the files are indeed encrypted, the attacker enumerates all user accounts available on the system and deletes the account of the user whose password is brute-forced at the beginning of this scenario. The attacker proceeds with deleting some directories as well as the system's backup files. Finally, the attacker stops a system service related to mail exchange.

3.4.4. Scenario 4: Network Attack

In Scenario 4, we assume that the attacker has already compromised the repository server within the DMZ and seeks to pivot into and compromise the entire network through targeted attacks against the firewall. The attack chain depicted in Fig. 8 starts with the preparation of an implant and an HTTPS listener, similar to the initial steps of Scenario 2. The attacker then re-uses the credentials of a compromised account on the repository server to first transfer a port knocking script to the firewall and then log in as a privileged user. The port knocking script is extracted and installed under the inconspicuous name *auditf* into the location where system binaries are stored; the archive is removed after extraction to reduce forensic traces. The attacker installs the port knocking tool by creating a service that triggers the script on start. Subsequently, they start the service before exiting the firewall and returning to the repository server.

From there, the attacker initiates the port knocking sequence, which causes the previously deployed port knocking tool on the firewall to down-

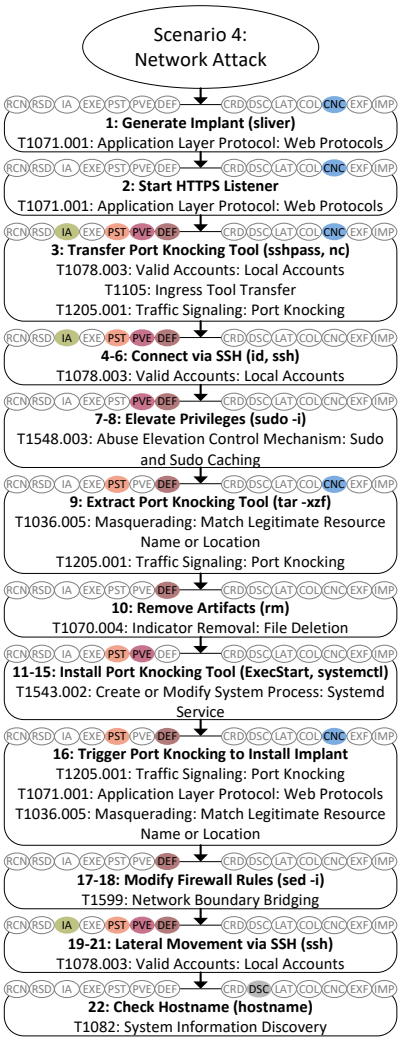


Figure 8: Attack paths of Scenario 4: Network Attack.

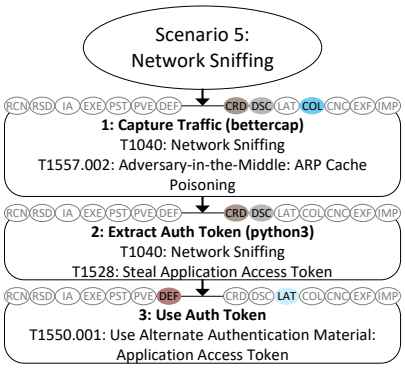


Figure 9: Attack path of Scenario 5: Network Sniffing.

load and run the implant prepared in one of the earlier steps. Through that implant, which provides remote command execution, the attacker then modifies and reloads the firewall rules so that hosts located in the DMZ are able to connect to hosts in the LAN, thereby effectively enabling to access the fileshare from the repository server. The attacker immediately leverages this ability and connects to the fileshare via SSH, where they print some system information to confirm that they reached the intended target.

3.4.5. Scenario 5: Network Sniffing

Figure 9 visualizes Scenario 5, which focuses on network sniffing. The attacker is assumed to have previously deployed a covert network access device in the LAN network, which provides remote access. Implanting this device has been carried out through physical access to one of the hosts within the network, e.g., through an insider who connects such a device to the USB or Ethernet port of an employee's desktop computer; since our focus lies on cyber attacks, this step is considered out of scope of the simulation of Scenario 5.

Through this device, the attacker launches the sniffing tool *bettercap* for ARP spoofing and to capture network traffic. After some time, we assume that a system administrator connected via LAN logs into the video server located within the DMZ. From the collected data, the attacker is able to extract the administrator's authentication token, which they subsequently re-use to make API calls on the video server.

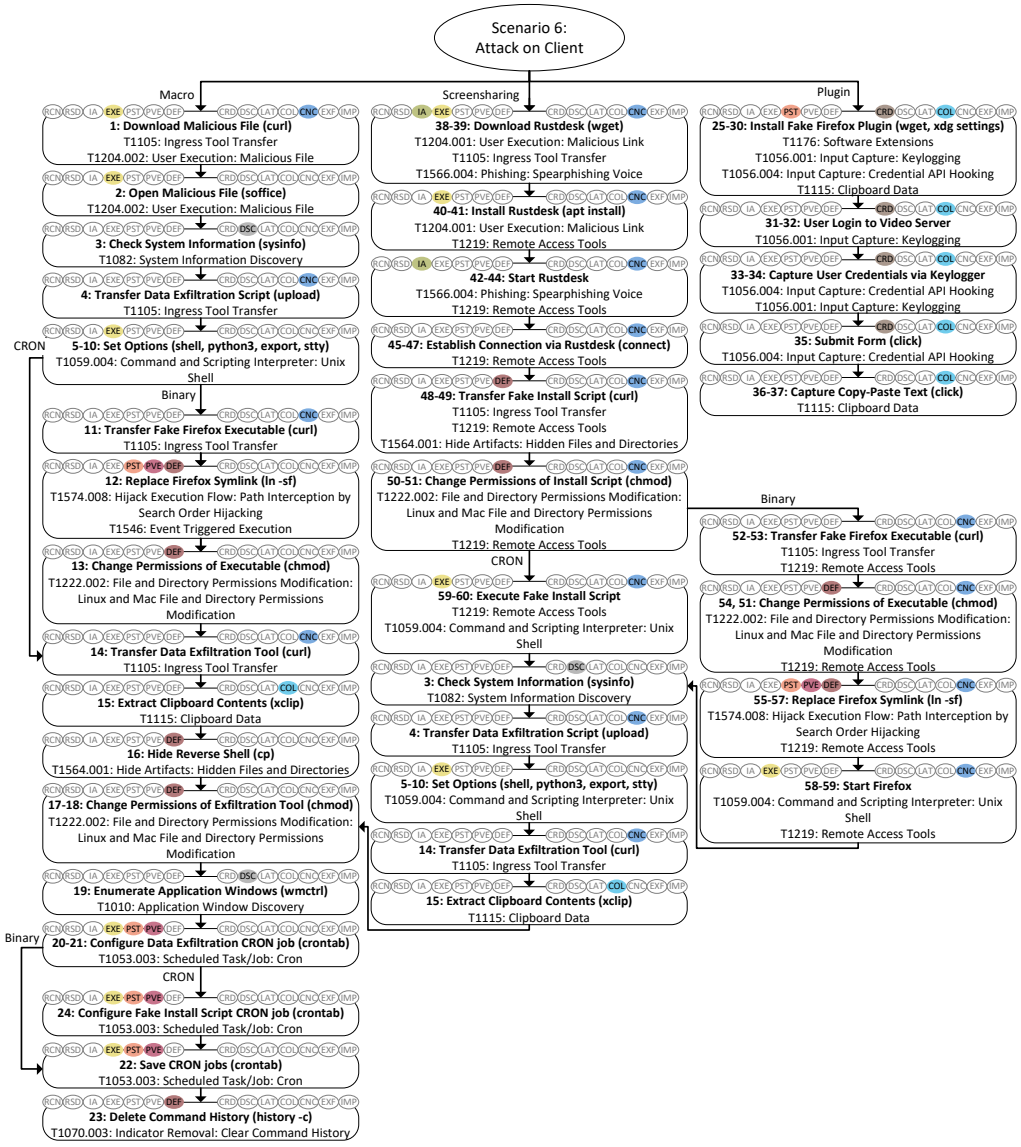


Figure 10: Attack path of Scenario 6: Attack on Client.

3.4.6. Scenario 6: Attack on Client

Scenario 6 covers three main attack chains targeting a client computer as visualized in Fig. 10. In contrast to previous scenarios, where the attacker primarily relies on tools or exploits of technical vulnerabilities, activities of the human user working on the targeted client computer play a key role for the initial intrusion. Thereby, we assume that the user carries out these activities purposefully but without realizing their malicious nature, for example, through social engineering, where the attacker calls the victim by telephone, pretends to be a technician from the organization's helpdesk, and persuades the user to follow their instructions. Note that in order to avoid that any artifacts from user simulation end up in the log data, we assume that the user remotely accesses their client computer through Virtual Network Computing (VNC). We argue that such remote access software is common in many organizations and does not affect the realism or validity of this attack chain.

Our scenario involves multiple variants; we start our description with the shortest attack chain. (i) *Plugin.* The attacker contacts the user and informs them about the necessary installation of a plugin for the *Firefox* browser as part of a company policy. For the purpose of this scenario, we assume that the user is a web engineer who uses the browser in developer mode, which is necessary for installation. Unknown to them, the plugin is in fact a keylogger crafted by the attacker, who hosts it on their web server under the unsuspecting domain *dailynews-wire.com*. The attacker talks the user through the process of downloading and installing the malicious plugin as well as configuring and restarting the browser. After ending the call, the attacker only needs to wait for the user to type their username and password when logging into any website. As part of our scenario, we assume that the user navigates to the ZoneMinder website and enter their credentials in the login form, which is captured by the keylogger and transmitted to the attacker. As a final step of this attack chain, the user copies and pastes the contents of the browser's search bar; such clipboard data is also captured and transmitted by the plugin.

The other attack chain in this scenario starts with two variants for initial access. (ii) *Macro.* The attacker informs the victim user about a document containing information on organizational policies. Subsequently, the attacker instructs the victim user to open a terminal and download that document from *dailynews-wire.com*, which appears as a valid domain but is in fact hosted by the attacker. Likewise, the downloaded file seems to

be a legitimate text document to the unsuspecting user based on its name and contents. However, the attacker previously implanted a malicious macro into the file that is triggered when opening it with the text editing software *LibreOffice*[28]. When activated, this macro establishes a reverse-shell that provides the attacker with remote access to the system. (iii) *Screensharing.* In this variant, the attacker convinces the user to download and install the legitimate tool *rustdesk*[29] under the disguise of helpdesk software. This tool enables to share and remotely control screens, which supposedly allows the attacker to fix some technical issues on the client's computer. As part of this procedure, the attacker instructs the user on how to configure the tool, shares necessary IP addresses and passwords, and tells them to accept the incoming connection when the attacker activates the same tool on their side. Even though this provides the attacker with remote access on the target system, the remote desktop software makes all actions visible on the user's screen, which means that the attacker's capabilities are limited to non-suspicious activities. In addition, their access is only temporary, because the user needs to manually confirm another connection after closing this session. In order to obtain persistent access, the attacker downloads a reverse-shell from their own typosquatted domain *facebock.com*, stores it as a hidden file, and changes its permissions. We assume that these activities do not trigger any suspicions, as the user fails to notice the misspelled domain and expects the alleged technician to diagnose and resolve issues on their computer.

For the remaining attack chains of *Macro* and *Screensharing* variants we employ similar attack techniques, but slightly vary their order. For brevity, we only explain each step once and refer to Fig. 10 for their position in the attack chain of the respective variant.

Both the reverse-shell established through the malicious macro as well as the remote desktop software only provide temporary access that only lasts until the system is rebooted or the session is terminated. We consider two variants how the attacker obtains persistence on the target system. (i) *Browser Binary.* The attacker downloads a binary for a fake executable for the Firefox browser from their own domain, which opens a reverse-shell before starting the browser as usual. Then, they redirect the symbolic link (symlink) of the actual Firefox browser to point to the fake executable. The attacker

[28] `https://www.libreoffice.org/`
[29] `https://rustdesk.com/`

thus obtains remote access every time when the user opens the browser, even after the system is restarted. (ii) *CRON*. The attacker creates a CRON job that invokes a previously downloaded script that in turn establishes a remote-shell. Note that this CRON job is configured and started at the end of the attack chain alongside other CRON jobs. From that point on, the script is executed every 5 minutes even after the system reboots.

Independent of the aforementioned variant, the attacker runs the *sysinfo* command to validate that the remote-shell works as expected and to obtain some information about the system. The attacker's next objective is to extract data from the compromised system. To this end, they transfer and execute *VeilTransfer*[30], a tool that provides several exfiltration techniques and is commonly used in security testing. The attacker uses a script to archive and transfer directories containing potentially sensitive data, in particular, the mail inbox of the Thunderbird application as well as session restore files, key database, and saved credentials of the Firefox browser. At the end of the attack chain, the script is configured as a CRON job that is executed every 10 minutes.

As an additional exfiltration activity, the attacker extracts clipboard contents and enumerates open application windows as part of collection and discovery activities. As a last step of the attack chain, the attacker clears the command history, which makes it more difficult to trace their activities in case that forensic analyses are carried out once the user becomes aware of the attack.

3.4.7. Scenario 7: Docker Attack

As visible in the attack chain depicted in Fig. 11, Scenario 7 starts with an enumeration of the corporate DNS servers identical to the first step of Scenario 1 (cf. Sect. 3.4.1). In this case, however, the scan reveals a mail service that is present in the network. The attacker decides to enumerate user names stored on the server using the *smtp-user-enum*[31] tool, which yields several results. The attacker chooses user Alice at random and proceeds to brute-force the password of the user's IMAP-account with *hydra*[32]. Note that in Scenario 3, the same tool is used to brute-force an SSH login (cf. Sect. 3.4.3). We also point out that the AttackMate playbook depicted in Fig. 4

[30] `https://github.com/infosecn1nja/VeilTransfer`
[31] `https://github.com/cytopia/smtp-user-enum`
[32] `https://github.com/vanhauser-thc/thc-hydra`

Figure 11: Attack path of Scenario 7: Docker Attack.

illustrates how these initial three steps of the attack chain are implemented for automated execution.

After successfully retrieving the password, the attacker sends a simple web probe to the server and realizes that this host is providing the open-source file sharing platform *Nextcloud*[33] in addition to the mail service. In an attempt to gain access to that system, the attacker executes a Metasploit module that exploits a vulnerable *Nextcloud* application that allows authenticated users to add workflows for command execution[34]. In our scenario, this vulnerable application is present on the platform, which means that the attacker is able to successfully execute the exploit using Alice's mail address and password for authentication.

Through the newly gained shell access, the attacker checks system user information to confirm that they are a non-privileged system user within a containerized environment. At this point, the attacker's primary objective is to escape from the container and gain access on the host running the containers. For the purpose of this scenario, we assume that the Docker API is exposed and accessible from within the container due to weak configurations. This allows the attacker to gather information from the running Docker containers and to enumerate Docker networks. Given this information, the attacker is able to escape the container by temporarily deploying a privileged container through the Docker API, mounting the *etc* directory into that new container, and placing a CRON job that downloads and installs an implant for reverse-shell access into the mounted directory. Once the CRON job triggers, the attacker utilizes the activated implant to access the host system, in particular, to enumerate files and processes as well as to download credential information.

4. Data Set Analysis

This section provides an analysis of the collected log data sets. We first present an overview of the data and then examine how individual attack steps manifest in the collected logs.

[33] `https://github.com/nextcloud`

[34] `https://www.rapid7.com/db/modules/exploit/unix/webapp/nextcloud_workflows_rce/`

4.1. Data Preparation

This section describes the data pre-processing steps applied to isolate attack manifestations. We also provide an overview of these manifestations by visualizing log events as timelines and summarizing the covered attack techniques.

4.1.1. Log Event Filtering

Analysis of attack manifestations benefits from isolating log events that are direct consequences of attacker activity. As described in Sect. 3.1, we aim to achieve such isolation by operating on an idle infrastructure without user interaction and by waiting 15 minutes after system boot before initiating attack execution to avoid interference from startup-related processes. Despite these precautions, an initial inspection of the collected data revealed that even idle systems continuously generate a substantial number of log events that inevitably overlap with attack execution. These background events primarily stem from processes that operate independently of user activity, including periodically scheduled system jobs, time synchronization mechanisms, and sporadically occurring system errors such as timeouts.

To mitigate this issue, we apply a filtering approach that removes log events that correspond to normal system behavior with high probability. Specifically, we consider the ten-minute interval preceding the start of AttackMate execution as a baseline period that captures log activity of an idle system. We compile log events occurring during this period across all hosts and remove timestamps, counters, and random values from these events using regular expressions. We then filter out all log events from the attack execution phase that are identical to entries in this reference set. We acknowledge that some attack steps may also trigger log events that are identical to those observed during normal system operation and may therefore unintentionally be removed by this process. While the informative value of the contextual occurrence of such events could be useful, we argue that their value for analysis is limited compared to those events that remain due to their syntax or parameters.

Even though this automated filtering approach effectively removes a substantial part of normal background events, we still observe certain events that contain highly variable parameters and are thus difficult to normalize with regular expressions. This particularly affects audit logs, for which we thus apply additional manual filtering based on process identifiers, e.g., to

remove events related to intrusion detection activity. We refer to our open-source code repository for more information on event filtering. We emphasize that the analyses presented in the following sections only consider the filtered data sets; however, we provide both the filtered and unfiltered data sets in our public repository, allowing researchers to select the version that best fits their use case.

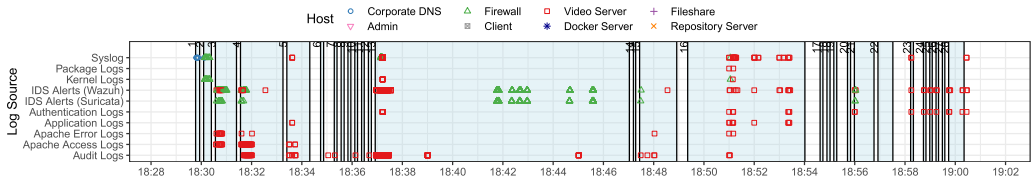
4.2. Timelines

Figure 12 visualizes the attack manifestations of Scenarios 1–7 based on the filtered data sets described in the previous section. For brevity, we display only a single attack path per scenario, using arbitrarily selected variants. Shaded intervals indicate the start and end times of individual attack steps; the numbers above these intervals correspond to the attack step identifiers used in Figs. 5–11. Each point in the plots represents the occurrence of a log event or alert, where the vertical axis indicates the source of the event and the color denotes the host on which the event was generated.

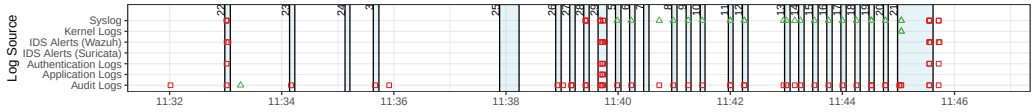
The visualizations show that log events accumulate during specific attack steps, with some steps triggering events across multiple log sources or on several hosts. We consider this as an indicator that our filtering procedure successfully removes most events corresponding to normal background activity, while preserving events that are direct consequences of attacker behavior. Furthermore, the figures show that the majority of log events are generated on hosts that are the primary targets of the respective scenarios, such as the video server in Scenario 1 and the client machine in Scenario 6. A small number of events occur outside attack intervals; these correspond either to background activity that was not filtered out or to attack-related events associated with steps that are not explicitly labeled. The figure also illustrates that certain attack steps do not result in any observable log events at all.

4.3. Coverage of Attack Tactics and Techniques

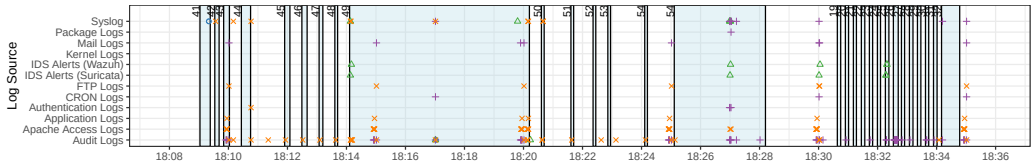
As outlined in Sect. 3.4, the attack scenarios are designed to represent coherent kill chains that collectively cover a broad range of techniques from the MITRE ATT&CK framework [46]. Table 3 summarizes the number of distinct techniques present in each scenario, grouped by their corresponding tactics. Note that techniques associated with multiple tactics are counted separately for each of them. The table shows that all scenarios span multiple tactics, while also revealing that certain scenarios emphasize specific tactics, such as Discovery in Scenario 1 or Impact in Scenario 3.



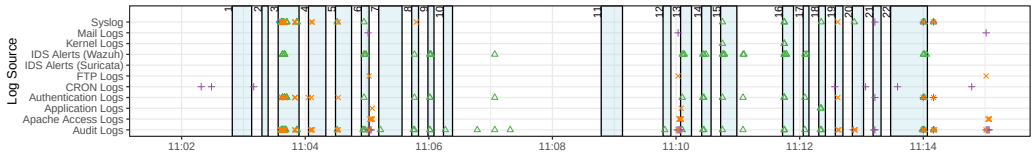
(a) Scenario 1: Video Server Exploit with the Autostart and Local Account variants.



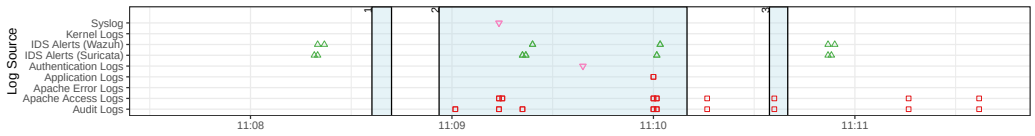
(b) Scenario 2: Linux Malware with the Rootkit variant.



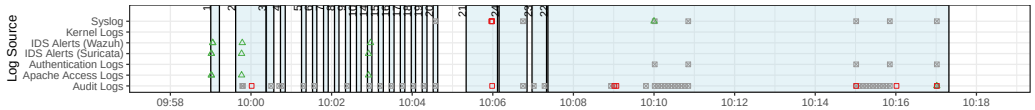
(c) Scenario 3: Lateral Movement with the VNC and Apt variants.



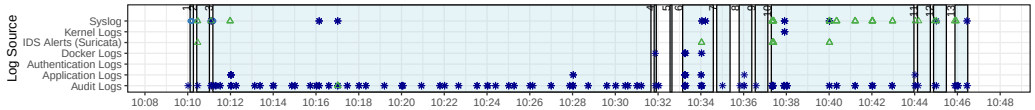
(d) Scenario 4: Network Attack.



(e) Scenario 5: Network Sniffing.



(f) Scenario 6: Attack on Client with the Macro and CRON variants.



(g) Scenario 7: Docker Attack.

Figure 12: Timelines of log event and alert occurrences. Shaded intervals indicate active attack steps.

Table 3: Number of attack techniques in scenarios

Tactic	Scenario 1	Scenario 2	Scenario 3	Scenario 4	Scenario 5	Scenario 6	Scenario 7	Total (Distinct)	MITRE ATT&CK Coverage
Reconnaissance	4	0	0	0	0	0	4	5	45.5%
Resource Development	0	0	0	0	0	0	0	0	0.0%
Initial Access	2	0	2	1	0	1	3	4	36.4%
Execution	2	1	3	0	0	3	3	6	35.3%
Persistence	7	3	4	3	0	4	4	13	56.5%
Privilege Escalation	7	3	3	3	0	3	2	9	64.3%
Defense Evasion	6	4	4	6	1	4	2	15	31.9%
Credential Access	2	1	3	0	3	1	2	7	41.2%
Discovery	16	8	6	1	1	2	4	20	58.8%
Lateral Movement	0	1	3	0	1	0	1	5	55.6%
Collection	0	3	2	0	1	2	0	8	47.1%
Command and Control	1	3	2	3	0	2	3	6	33.3%
Exfiltration	0	1	0	0	0	0	1	1	11.1%
Impact	0	0	6	0	0	0	0	6	40.0%
Total (Distinct)	35	24	28	10	4	16	22	81	37.5%

In addition to the per-scenario breakdown, we also report the total number of distinct techniques covered across all scenarios for each tactic, excluding duplicate techniques that appear in multiple scenarios or are associated with multiple tactics. To contextualize these numbers, we report the corresponding coverage relative to the complete set of techniques defined in the MITRE ATT&CK framework in the final column of the table.

Overall, the scenarios comprise a total of 81 distinct techniques, corresponding to 37.5% of all techniques defined in the MITRE ATT&CK framework and covering 16 out of the 19 most prevalent techniques mentioned across 667 cyber threat intelligence reports [50]. Several techniques are executed multiple times across different scenarios, using different methods and in varying contexts, which enables a broad analysis of their manifestations in log data. Specifically, 34 distinct techniques are part of at least two scenarios. Furthermore, some techniques are realized through multiple sub-techniques, which are labeled accordingly. When accounting these labels, the scenarios cover a total of 97 out of 475 distinct sub-techniques specified in MITRE ATT&CK.

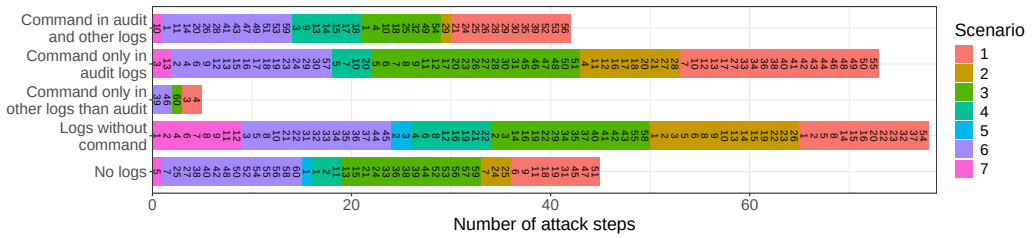


Figure 13: Observability of attacks and commands executed by the attacker in log data.

4.4. Cyber Attack Manifestations

This section analyzes how attack techniques manifest in system log data.

4.4.1. Observability of Attack Commands

Both manual log analysis by experts as well as LLM-based log interpretation rely heavily on the expressiveness of log event contents. Ideally, every action carried out by attackers generates log events that contain sufficiently granular information to make the underlying activity straightforward to understand. In practice, however, logs often capture only indirect consequences of actions rather than the actions themselves, or lack essential contextual information. To better understand how attacks manifest in log data, we investigate which attack techniques generate log events that contain explicit indicators of attacker activity.

To this end, we extract the commands executed by our attack emulation tool for each attack step and search the collected log data for occurrences of these commands. Specifically, we split each command into individual tokens on spaces and remove common stop-words (e.g., "sudo", "id") that yield too many matches in unrelated events. We consider an attack step to contain an explicit command indicator if at least one of the remaining tokens appears in any log events captured during the time interval where that attack step was captured.

Figure 13 summarizes the results of this experiment. Overall, a substantial fraction of attack steps generate log events that contain information about the executed command. Out of the 243 attack steps in our data set (cf. Sect. 3.4), 115 (47.3%) leave such indicators in audit logs. This result is largely explained by the fact that many attack chains involve direct command execution on compromised systems, for example as part of information gathering. Since audit logs record executed commands, they constitute a particularly rich source of information for log interpretation. Figure 14a shows

sample audit log events capturing the command "cat /etc/shadow", which the attacker uses to dump credential hashes. Note that in some cases, audit logs encode command information in hexadecimal form; in the provided example, decoding the "proctitle" field also reveals the executed command.

Other log sources, including syslog, access logs, authentication logs, etc., also contribute relevant information. In addition to audit logs, 42 attack steps (17.3%) leave explicit command indicators in other log sources, while 5 attack steps (2.1%) generate such indicators exclusively outside of audit logs. The latter primarily involves remote tools that are not executed directly on the target system. For example, the access logs shown in Fig. 15 correspond to scanning activity and contain the keyword "nikto", identifying the scanner used by the attacker from outside the network. At the same time, our analysis shows that 78 attack steps (32.1%) generate log events that do not directly reference the executed command, and 45 attack steps (18.5%) that do not involve any log events at all after filtering.

Importantly, the presence of command strings in log data does not imply that an attack step is straightforward to interpret. First, identifying relevant commands becomes challenging when they are concealed by large volumes of log events with diverse parameters. Second, correct interpretation often depends on contextual information. For instance, audit logs generally record executions of commands such as "chmod", which are useful to make attack scripts executable (e.g., Scenario 6, Step 13); however, the same command may also be applied on benign files as part of routine administrative activity. Distinguishing benign from malicious use thus requires additional context, such as affected files and directories, the involved user, the temporal context of the action, etc.

A key finding of our analysis is that attack manifestations depend not only on which command is executed, but also on how it is executed. Figure 14 illustrates this effect using three attack steps from different scenarios that all execute the same command: "cat /etc/shadow". In Scenario 3, the attacker executes the command via an interactive VNC session, resulting in audit logs that clearly expose both the command and its arguments (cf. Fig. 14a). In Scenario 7, the same command is executed through an implant, and the corresponding logs (cf. Fig. 14b) reveal only the accessed file path while obscuring the command itself. In Scenario 1, where the attacker interacts with the system through a reverse-shell, the command appears in authentication logs (cf. Fig. 14c). Since even identical commands may surface in different log sources and with varying levels of granularity depending on the

```
1 type=SYSCALL msg=audit(1765563116.496:15535): arch=c000003e syscall=59
      success=yes exit=0 a0=5583f2d27de0 a1=5583f2e33fe0 a2=5583f2e2ff80 a3=8
      items=2 ppid=4269 pid=4289 auid=4294967295 uid=0 gid=0 euid=0 suid=0
      fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts2 ses=4294967295 comm="cat" exe="/
      usr/bin/cat" subj=unconfined key="T1078_Valid_Accounts"
2 type=EXECVE msg=audit(1765563116.496:15535): argc=2 a0="cat" a1="/etc/shadow
      "
3 type=PATH msg=audit(1765563116.496:15535): item=0 name="/usr/bin/cat" inode
      =1568 dev=fc:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL
      cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
4 type=PATH msg=audit(1765563116.496:15535): item=1 name="/lib64/ld-linux-x86
      -64.so.2" inode=79850 dev=fc:01 mode=0100755 ouid=0 ogid=0 rdev=00:00
      nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
5 type=PROCTITLE msg=audit(1765563116.496:15535): proctitle=636174002
      F6574632F736861646F77
```

(a) Command visible as clear text and hexadecimal-encoded text in audit logs of Scenario 3, Step 45.

```
1 type=SYSCALL msg=audit(1768992356.109:5653): arch=c000003e syscall=257
      success=yes exit=7 a0=ffffffffffffff9c a1=c00046ec50 a2=80000 a3=0 items
      =1 ppid=3689 pid=3717 auid=0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0
      sgid=0 fsgid=0 tty=(none) ses=31 comm="tool" exe="/opt/tool" subj=
      unconfined key="T1087_Account_Discovery"
2 type=PATH msg=audit(1768992356.109:5653): item=0 name="/etc/shadow" inode
      =5154 dev=fd:01 mode=0100640 ouid=0 ogid=42 rdev=00:00 nametype=NORMAL
      cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
3 type=PROCTITLE msg=audit(1768992356.109:5653): proctitle="/opt/tool"
```

(b) Parts of command visible in audit logs of Scenario 7, Step 13.

```
1 Sep 22 18:58:45 videoserver sudo: webadmin : PWD=/home/webadmin ; USER=root
      ; COMMAND=/usr/bin/cat /etc/shadow
2 Sep 22 18:58:45 videoserver sudo: pam_unix(sudo:session): session opened for
       user root(uid=0) by (uid=1003)
3 Sep 22 18:58:45 videoserver sudo: pam_unix(sudo:session): session closed for
       user root
```

(c) Command visible in authentication logs of Scenario 1, Step 24.

Figure 14: Observability of the attack command for credential dumping (T1003.008) across scenarios.

```
1 192.42.1.174 - - [22/Sep/2025:18:30:44 +0000] "GET /old/ HTTP/1.1" 404 396
    "-" "Mozilla/5.00 (Nikto/2.1.5)"
2 192.42.1.174 - - [22/Sep/2025:18:30:44 +0000] "GET /oracle HTTP/1.1" 404 396
     "-" "Mozilla/5.00 (Nikto/2.1.5)"
3 192.42.1.174 - - [22/Sep/2025:18:30:44 +0000] "GET /oradata/ HTTP/1.1" 404
    396 "-" "Mozilla/5.00 (Nikto/2.1.5)"
4 192.42.1.174 - - [22/Sep/2025:18:30:44 +0000] "GET /order/ HTTP/1.1" 404 396
     "-" "Mozilla/5.00 (Nikto/2.1.5)"
5 192.42.1.174 - - [22/Sep/2025:18:30:44 +0000] "GET /orders/ HTTP/1.1" 404
    396 "-" "Mozilla/5.00 (Nikto/2.1.5)"
6 192.42.1.174 - - [22/Sep/2025:18:30:44 +0000] "GET /orders/checks.txt HTTP
    /1.1" 404 396 "-" "Mozilla/5.00 (Nikto/2.1.5)"
```

Figure 15: Excerpt of the log events generated as a consequence of scanning activities from Scenario 1, Step 3.

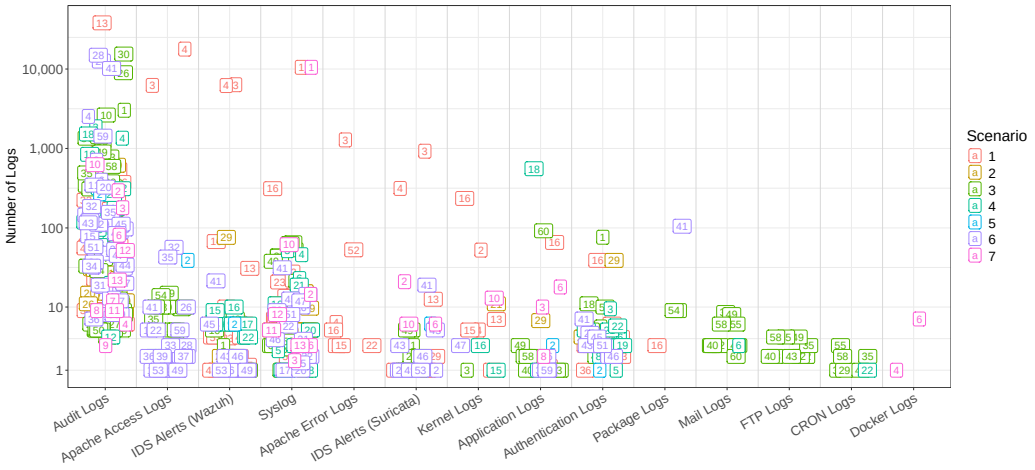


Figure 16: Log event frequencies observed in different log sources. The number in the tags reference the attack step of the respective scenario.

execution method, we conclude that no consistent mapping between attack techniques and specific log manifestations can be established.

4.4.2. Log Event Frequencies

Aside from the contents of individual log messages, analysts frequently monitor the frequency of log events, for example, through dashboards that aggregate event counts over fixed time intervals. Changes in event frequencies are commonly used as basic indicators of anomalies in system behavior, because significant deviations such as spikes that stand out from long-term baselines may point to system issues or malicious activity [51, 52].

Several attack techniques included in our data set generate large volumes

```
1 Dec 11 14:28:03 puppet sshd[3089]: Invalid user admin from 192.42.1.174 port
    33006
2 Dec 11 14:28:03 puppet sshd[3089]: Received disconnect from 192.42.1.174
    port 33006:11: Bye Bye [preauth]
3 Dec 11 14:28:03 puppet sshd[3095]: pam_unix(sshd:auth): check pass; user
    unknown
4 Dec 11 14:28:03 puppet sshd[3095]: pam_unix(sshd:auth): authentication
    failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.42.1.174
5 Dec 11 14:28:05 puppet sshd[3095]: Failed password for invalid user admin
    from 192.42.1.174 port 56768 ssh2
```

Figure 17: Failed authentication events generated as a consequence of a brute-force login from Scenario 3, Step 1.

of log events. It is difficult to determine, however, which of these techniques would trigger suspicion in operational environments, because this kind of detection depends on the individual event count baselines. Since our data set does not include simulations of benign workload behavior, we only compare the frequencies of log events generated by different attack steps with each other. Figure 16 visualizes the number of log events produced per attack step across log sources. As shown in the figure, audit logs account for the largest share of events. The main reason for this is that a large number of attack steps directly interact with systems and thus trigger audit logging, causing anything from a few to several thousands of events. Thereby, the system vulnerability scan performed in Step 13 of Scenario 1 produces the highest volume of audit logs, generating more than 40,000 events.

In contrast, most other log sources typically do not generate more than 100 events per attack step, often substantially fewer. Notable exceptions to this tendency are attack steps involving network-based scanning activities. For example, the DNS enumeration in Step 1 of Scenario 1 results in a high number of syslog entries on the DNS server, the network scan in Step 2 of Scenario 1 generates numerous system logs in the firewall, and the network vulnerability scan in Step 3 of Scenario 1 produces a large volume of access logs. An excerpt of the latter is shown in Fig. 15, which depicts a rapid succession of requests corresponding to entries from a wordlist. Similar log characteristics can be observed for the brute-force login executed in Step 1 of Scenario 3. Figure 17 shows an excerpt of the corresponding logs that document a series of failed authentication events taking place within a few seconds.

4.4.3. System Performance Metrics

```
[2025-09-22 18:37:00] [info] write_log values:
videoserver_novalocal.load.load.shortterm 0.1
[2025-09-22 18:37:10] [info] write_log values:
videoserver_novalocal.load.load.shortterm 0.24
[2025-09-22 18:37:20] [info] write_log values:
videoserver_novalocal.load.load.shortterm 1.12
[2025-09-22 18:37:30] [info] write_log values:
videoserver_novalocal.load.load.shortterm 1.71
[2025-09-22 18:37:40] [info] write_log values:
videoserver_novalocal.load.load.shortterm 1.44
```

Figure 18: System metric captured with collectd.

Log collection frameworks such as collectd[35] capture system performance metrics over time. The collected measurements provide insights into system utilization at given points in time and are particularly useful for correlating system activities with changes in system performance, for example, when programs with high computational workload are executed. Common categories of metrics include CPU, file system, disk, entropy, network interfaces, interrupt counters, load, memory, processes, and swap usage; each category may comprise multiple individual measurements. Figure 18 shows an excerpt of collectd logs capturing short-term system load in intervals of ten seconds. The displayed measurements indicate a noticeable increase in system load, which is caused by the system vulnerability scan (Scenario 1, Step 13).

Figure 19 visualizes the evolution of system performance metrics captured during the entire simulation period of Scenario 1, including system load, CPU utilization of user-space processes, the number of disk read operations, total memory usage, the number of received network packets, and the number of running processes. Note that for visualization purposes, all metrics are scaled to a common range. The figure reveals that certain attack steps have a measurable impact on system performance. Corresponding to the logs displayed in Fig. 18, the system vulnerability scan executed in Step 13 leads to a sharp increase in CPU utilization, disk read activity, and system load. These results demonstrate that attack steps do not only manifest through generation of log events but also impact measurements in continuously generated system performance metrics.

[35] `https://collectd.org/`

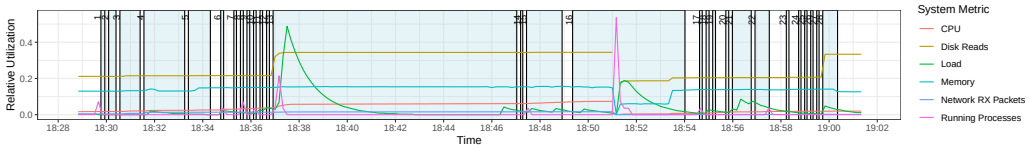


Figure 19: Timelines of system performance metrics collected from the video server in Scenario 1 (Autostart, Local Account).

```
{
  "rule": {
    "level": 10,
    "description": "Auditd: Device enables promiscuous mode.",
    "id": "80710"
  },
  "full_log": "type=ANOM_PROMISCUOUS msg=audit(1765797045.372:6848): dev=
      ens3 prom=256 old_prom=0 auid=4294967295 uid=0 gid=0 ses=4294967295",
  "location": "/var/log/audit/audit.log"
}
{
  "rule": {
    "level": 3,
    "description": "Successful sudo to ROOT executed.",
    "id": "5402"
  },
  "full_log": "Dec 15 11:10:45 inetfw sudo: root : TTY=pts/1 ; PWD=/usr/bin
      ; USER=root ; COMMAND=/usr/bin/systemctl start auditf.service",
  "location": "/var/log/auth.log"
}
```

Figure 20: Alerts triggered by Wazuh IDS when installing the port knocking tool in Scenario 4, Step 15.

4.4.4. Intrusion Detection Alerts

Certain types of log events are considered suspicious because they are commonly produced as consequences of malicious or otherwise undesired system activities. Similarly, specific behavioral patterns observable in network traffic can indicate potential attacks. Intrusion Detection Systems (IDS) analyze system logs and network traffic for matches against predefined detection rules and signatures, triggering alerts when such patterns are observed. As described in Sect. 3.1.3, our experimental setup includes the host-based IDS Wazuh and the network-based IDS Suricata, both configured to use their respective publicly available default rule sets.

Figures 20 and 21 illustrate sample alerts generated by these systems. Figure 20 shows alerts triggered by Wazuh during Step 15 of Scenario 4, in which the attacker installs a port-knocking tool. The first alert is a high-severity alert (level 10) indicating that a network interface has been switched into promiscuous mode, which is typically associated with network monitoring and therefore considered security-critical. The second alert has a sub-

```
1 12/11/2025-14:42:48.133080  [**] [1:2034567:1] ET HUNTING curl User-Agent to
      Dotted Quad [**] [Classification: Potentially Bad Traffic] [Priority:
      2] {TCP} 192.168.100.23:49346 -> 192.42.1.174:80
2 12/11/2025-14:42:48.133803  [**] [1:2019240:14] ET POLICY Executable and
      linking format (ELF) file download Over HTTP [**] [Classification:
      Potential Corporate Privacy Violation] [Priority: 1] {TCP}
      192.42.1.174:80 -> 192.168.100.23:49346
```

Figure 21: Alerts triggered by Suricata IDS when downloading ransomware in course of Scenario 3, Step 25.

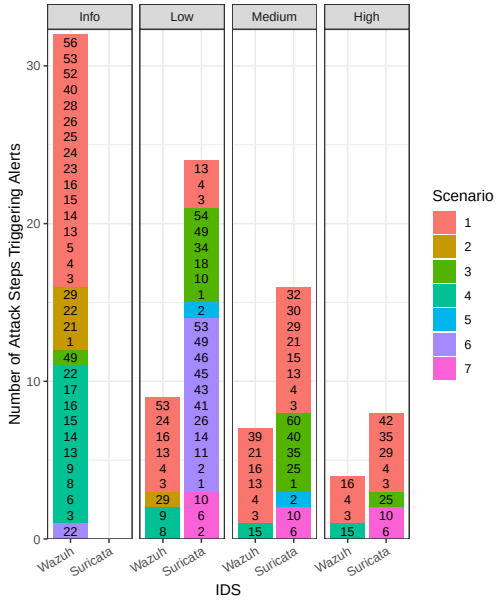


Figure 22: Severities of intrusion detection alerts across scenarios and attack steps.

stantially lower severity (level 3) and records the execution of a privileged command. The main reason for the low severity is that these commands are common in benign administrative workflows. Figure 21 presents two alerts generated by Suricata during Step 25 of Scenario 3, in which the attacker downloads ransomware. The first alert has medium severity (priority 2) and flags an HTTP request to an IP address, which is in our case controlled by the attacker. The second alert has the highest severity level (priority 1) and is triggered because an executable file is downloaded over unencrypted HTTP. Note that Suricata and Wazuh use different severity scales, which necessitates normalization for comparative analysis.

We analyze all IDS alerts triggered during the execution of our attack sce-

narios. Figure 22 summarizes the attack steps that result in alerts and groups them by severity. To enable comparability across IDS, we map Wazuh's severity levels onto the three-point scale used by Suricata as follows: levels 4–6 correspond to Low, levels 7–9 to Medium, and level 10 and above to High. Wazuh alerts with severity of level 3 or lower are treated as Informational, as they do not necessarily indicate malicious activity. Note that single attack steps may trigger multiple alerts across different severity levels (cf. Figs. 20 and 21) and from both IDSs at the same time. For example, the scanning activities in Steps 3 and 4 of Scenario 1 generate alerts from both Wazuh and Suricata, spanning all severity categories. Overall, the results show that Suricata triggers alerts for a larger number of attack steps in the Low-High severity ranges, whereas Wazuh predominantly generates informational alerts across many steps.

Our analysis shows that out of the 198 attack steps that produce log data, only 43 trigger alerts with a severity level of Low-High in at least one of the two IDS. An additional 20 attack steps trigger informational alerts in Wazuh. Conversely, 155 attack steps remain undetected by either IDS when relying on default detection signatures. These results highlight the limitations of signature-based intrusion detection in capturing a wide range of attacker activities and underscore the importance of complementary analysis approaches, such as methods for anomaly-based detection or LLM-based log interpretation.

5. LLM-based Interpretation

This section presents an illustrative evaluation of our data set in the context of log interpretation using LLMs.

5.1. Experimental Setup

In this section, we summarize the setup of our experiment.

5.1.1. Task & Model

We task an LLM with interpreting log data in a zero-shot setting, i.e., without providing any training samples. The goal of this experiment is to approximate a realistic usage scenario in which security analysts copy and paste log excerpts into an LLM to obtain a fast and automated first assessment

that complements manual analysis [9]. For this experiment, we use ChatGPT[36], a model that has also been employed for log interpretation in prior work [2, 6]. Moreover, we set the model's temperature to zero to minimize hallucinations.

5.1.2. Prompting

For each of the 198 attack steps that generate log data, we submit one prompt to ChatGPT. Thereby, each prompt starts with a task description instructing the model to (i) classify the provided log data according to the MITRE ATT&CK framework and return the top 10 most relevant techniques in descending order, (ii) estimate whether the logs correspond to malicious or benign behavior on a 7-point likert scale, and (iii) provide a short explanation on the reasoning behind the assessment.

Following this task description, we include log data collected from all hosts and log sources involved in the respective attack step. To avoid excessive prompt lengths, we limit the number of logs per host–source pair at ten lines. These lines are sampled at random positions within the corresponding log file. We repeat this process five times per attack step with new random samples of log lines in order to mitigate the effects of sampling randomness and to assess the consistency of resulting classifications. Figure 23 shows an exemplary prompt, including both the instruction preamble and selected access and audit logs from the video server during the ZoneMinder exploit (Scenario 1, Step 5). The logs depict the request to the web server that transmits the malicious payload, a part of the subsequent execution of the payload on the system itself, as well as a warning message about a missing event from the ZoneMinder application. Note that we simplify some logs and remove less relevant log sources for brevity.

5.2. Results

This setup presents the results obtained from our experiment.

5.2.1. Qualitative Examples

Figure 24 showcases responses for attack steps that are correctly interpreted. Figure 24a shows the response to the prompt in Fig. 23. Based on the explanation generated by the LLM, it becomes evident that the model

[36]We use ChatGPT version 5.2, which is the most recent version available at the time of conducting this experiment.

```
You are a MITRE ATT&CK TTP classification expert. Your task is to classify the following system log data. You are provided with samples from one or more hosts and one or more log sources that are captured during execution of one specific MITRE ATT&CK technique. Always output a valid JSON object with the following fields:
 - "techniques": A list of top 10 ATT&CK techniques that are most likely related to the sample logs, sorted in descending order. Only print the ID of the techniques without any other descriptions.
 - "confidence": An estimate for the certainty that the logs indicate an actual attack rather than normal system or user activity. Provide one of the following estimates: "Certain: Attack", "Almost Certain: Attack", "Somewhat Certain: Attack", "Neutral", "Somewhat Certain: Normal", "Almost Certain: Normal", "Certain: Normal"
 - "explanation": A brief explanation (1-2 sentences) why you think that the samples correspond to attacks or normal behavior, e.g., by pointing to specific artifacts or properties of the logs.

videoserver:
  logs/access.log.1:
    192.42.1.174 - - [24/Sep/2025:11:31:05 +0000] "GET /zm/index.php HTTP/1.1" 200 8978 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0"
    192.42.1.174 - - [24/Sep/2025:11:31:05 +0000] "POST /zm/index.php HTTP/1.1" 302 756 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0"
  logs/log/zm/web_php.log:
    9/24/25, 11:31:12 AM UTC.759462 web_php[2639].WAR [192.42.1.174] [No event returned from TriggerOn] at /usr/share/zoneminder/www/includes/Monitor.php line 678
  logs/log/audit/audit.log:
    type=SYSCALL msg=audit(1758713480.172:4453): arch=c000003e syscall=59 success=yes exit=0 a0=55df1de3c000 a1=55df1de3bf88 a2=55df1de3bfa0 a3=8 items=4 ppid=2688 pid=2689 auid=4294967295 uid=33 gid=33 euid=33 suid=33 fsuid=33 egid=33 sgid=33 fsgid=33 tty=(none) ses=4294967295 comm="which" exe="/usr/bin/dash" subj==unconfined key="T1166_Seuid_and_Setgid" ARCH=x86_64 SYSCALL=execve AUID="unset" UID="www-data" GID="www-data" EUID="www-data" SUID="www-data" FSUID="www-data" EGID="www-data" SGID="www-data" FSGID="www-data"
    type=EXECVE msg=audit(1758713480.172:4453): argc=3 a0="/bin/sh" a1="/usr/bin/which" a2="python"
```

Figure 23: Exemplary LLM prompt showing a task description and logs from Scenario 1, Step 5.

```
"techniques": ["T1059.004", "T1059.006", "T1082", "T1518.001", "T1046", "T1190", "T1105", "T1505.003", "T1078", "T1036"],
"confidence": "Somewhat Certain: Attack",
"explanation": "A web request to ZoneMinder (/zm/index.php) is followed by audit logs showing the web server account (uid/gid 33, www-data) spawning a shell to run command discovery (\"/bin/sh /usr/bin/which python\"), which is atypical for normal web browsing and suggests command execution via a web application. The ZoneMinder warning tied to the same source IP and the process execution from a web context increases likelihood of exploitation of a public-facing app."
```


(a) ZoneMinder exploit in Scenario 1, Step 5.


```
"techniques": ["T1548.003", "T1003.008", "T1078", "T1059.004", "T1087.001", "T1106", "T1068", "T1555", "T1552.001", "T1036"],
"confidence": "Almost Certain: Attack",
"explanation": "Logs show user \"debian\" invoking sudo to run \"cat /etc/shadow\" as root, which is a common precursor to credential dumping (reading local password hashes) and matches Wazuh's mapping to sudo-based privilege escalation (T1548.003). Access to /etc/shadow via elevated privileges is rarely needed in routine operations and strongly indicates malicious credential access activity."
```


(b) Credential dumping in Scenario 1, Step 24.


```
"techniques": ["T1595.002", "T1190", "T1083", "T1590.001", "T1590.004", "T1592.002", "T1213", "T1110", "T1505.003", "T1070.004"],
"confidence": "Almost Certain: Attack",
"explanation": "The logs show automated web probing with Nikto user-agent and many 404/invalid-URI requests for common/admin paths and CGI scripts, plus explicit traversal/LFI-style attempts to access sensitive files like /etc/passwd, boot.ini, and /system32/, consistent with vulnerability scanning and exploitation attempts against a public-facing web server."
```


(c) Wordlist scanning in Scenario 1, Step 3.

Figure 24: Samples of correct LLM-based classification and interpretation of logs from attack steps.

correctly connects the web request observed in the access logs with the command executed by the web application process recorded in the audit logs through temporal correlation, and identifies this behavior as unusual. Furthermore, the model links the ZoneMinder application warning message via IP-based correlation and interprets the event as corroborating evidence of potentially malicious activity. This form of multi-source reasoning resembles the analytical process expected from a human security analyst. The ground truth labels of the corresponding attack step, namely *Exploitation of Public-Facing Application (T1190)* as well as *Command and Scripting Interpreter (T1059)*, appear in the first and sixth positions of the top predictions, indicating a strong semantic understanding of both the meaning of the log content and the associated attack techniques. Figure 24b demonstrates the interpretation of credential dumping; relevant logs are depicted in Fig. 14c. The

response shows that the correct technique and sub-technique (*T1003.008*) appear at position two in the ranked list of predictions. The prediction for the top-ranked technique is likely influenced by the presence of a Wazuh alert referencing *Sudo Caching for Privilege Escalation (T1548.003)*. Again, the explanation of the underlying cause of the logs and their relevance to specific malicious activities are on point. Figure 24c shows the interpretation of vulnerability scanning, with log excerpts shown in Fig. 15. In this case, the LLM correctly identifies the scanning tool via the user-agent string as well as the high number of requests, thus leveraging both event content (cf. Sect. 4.4.1) and frequency (cf. Sect. 4.4.2). Once more, the top prediction matches the ground truth sub-technique *Active Scanning: Vulnerability Scanning* (T1595.002).

5.2.2. Quantitative Evaluation Results

We evaluate classification performance by comparing the predicted techniques with the ground truth labels across all five runs of the 198 attack steps. Thereby, we only consider matches at the technique level since the model was not instructed to return sub-technique identifiers. We record the rank of the highest correct prediction, where any of the ground-truth techniques appears in the top 10 list.

Figure 25 visualizes the results, showing both the rank of the highest matching technique (positions #1–#10 or not in top 10) and the corresponding likert-scale confidence estimates. For visualization purposes, we split the results into the top, middle, and bottom thirds with respect to attack step classification accuracy. The figure shows that the top third of the attack steps are classified with high accuracy, yielding correct technique predictions almost always at position #1 or #2. The middle third exhibits adequate results where a matching technique is found within the top 10 predictions. The bottom third contains hardly any correct predictions within the top 10.

Confidence estimates regarding maliciousness vary substantially across attack steps and do not always correlate with classification accuracy. We observe no tendency for attack steps from particular scenarios to be easier or harder to classify than others, indicating that all scenarios are equally suitable for this type of evaluation.

5.2.3. Impact of Manifestation Characteristics

Finally, we investigate the relations between different types of attack manifestations and the LLM's classification performance. To this end, we cor-

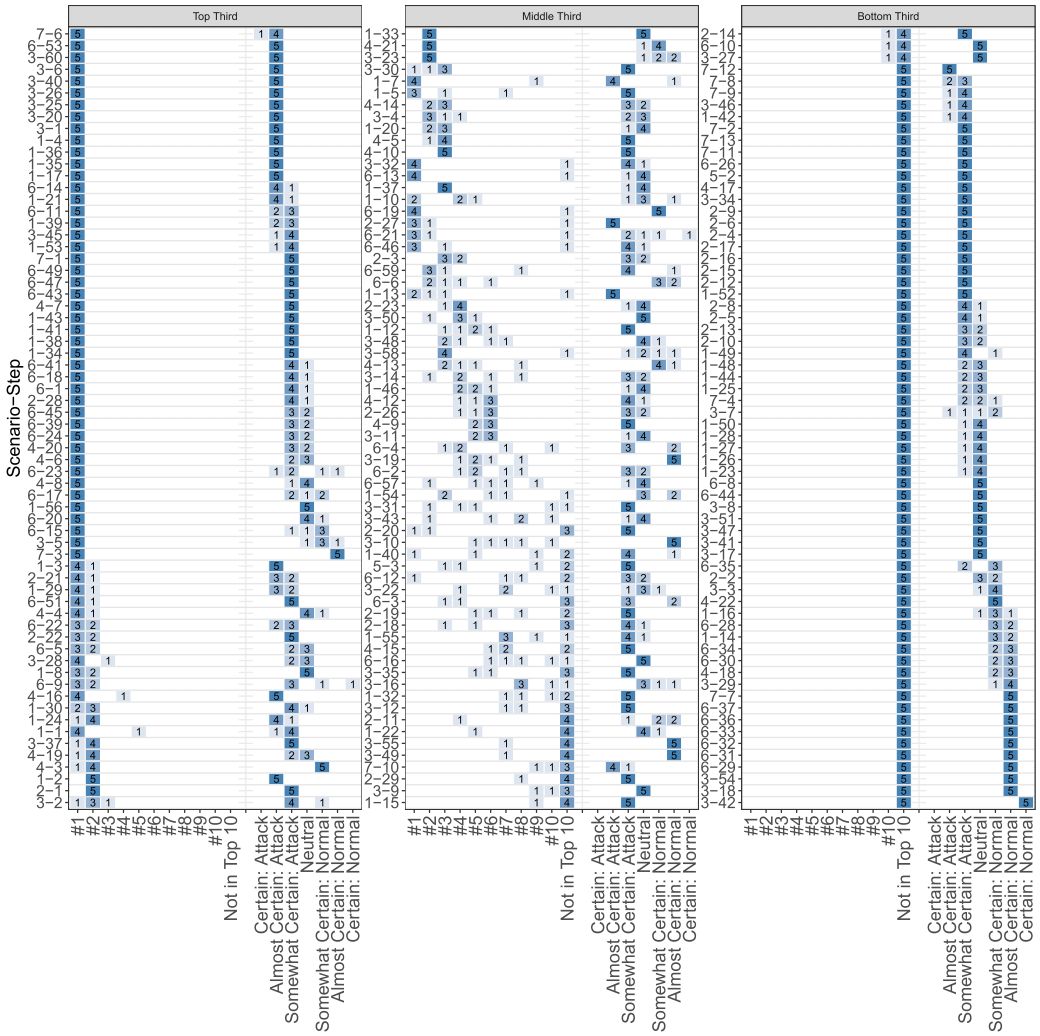
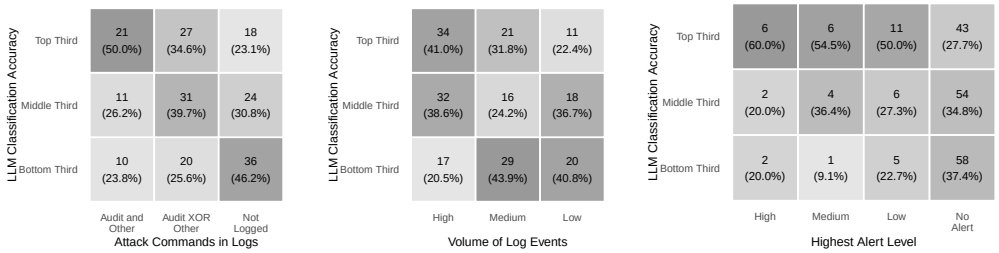


Figure 25: Presence of ground-truth labels among the top-10 predicted attack techniques and estimation of benign versus malicious origin.



(a) Observability of commands.

(b) Log event frequencies.

(c) Highest alert severities.

Figure 26: Contingency tables of LLM classification accuracies and attack manifestations.

relate these top, middle, and bottom third of classification results (cf. Fig. 25) with previously analyzed manifestation characteristics, namely command observability (cf. Sect. 4.4.1), event frequency (cf. Sect. 4.4.2), and presence of IDS alerts (cf. Sect. 4.4.4).

Figure 26 presents the results in the form of contingency tables. Figure 26a shows that classification accuracy is highest when attack commands are visible in both audit logs and other log sources, lower when commands appear in only one of these sources, and lowest when no command information is present in the logs analyzed by the LLM. Figure 26b indicates that attack steps generating large numbers of log events (> 1000 in audit or > 100 in other log sources) are more accurately classified than those with moderate (> 100 in audit or > 10 in other log sources) or lower event volumes. This may be attributable to the presence of stronger indicators in high-frequency attacks (cf. Fig. 15), a higher likelihood of relevant logs being sampled into the prompt, or the informative nature of frequency patterns themselves. Finally, Fig. 26c shows that most attack steps triggering IDS alerts fall into the top third of classification accuracy. This can be attributed either to the additional semantic information contained in the alerts themselves or to the fact that these attack steps are inherently more straightforward to identify as such and thus easier to interpret.

6. Discussion

In this section, we discuss the implications of our findings. We subsequently outline limitations of our work and derive recommendations for future research.

6.1. Implications

This paper introduces a new data set of cyber attack manifestations in system log data and provides an empirical analysis of their characteristics. Our results indicate that the majority of attack steps leave distinct traces in system logs that allow to reconstruct individual actions. Our analysis suggests that attack executions can be broadly grouped into at least four types of manifestations. First, attacks that leave no observable manifestations in common sources for system log data, which may occur for certain types of interactions as well as for preparatory steps that take place outside the monitored systems or network. Second, direct manifestations that expose executed commands. Across most scenarios, information about executed commands is present in the logs, despite the fact that attackers employ different execution mechanisms, including SSH (Scenarios 1, 3, and 4), implants (Scenarios 2 and 4), reverse-shells (Scenarios 1 and 3), VNC (Scenario 3), and screen sharing software (Scenario 6). While these mechanisms differ substantially in how attackers interact with compromised systems, our examples show that command information is often still observable. At the same time, the execution method strongly influences the granularity and visibility of these traces, with certain approaches effectively obscuring command-level details. Third, manifestations that do not reveal executed commands but are generated as direct consequences of malicious actions. Although less explicit, such events can still serve as strong indicators of adversary behavior, particularly when they stand out due to unusual parameters (e.g., events indicating system configuration changes) or high event frequencies (e.g., scanning activities). Fourth, indirect manifestations, such as changes in system performance metrics. These manifestations are generally more difficult to relate to specific attack techniques; nonetheless, unexpected changes of these metrics may be useful to detect undesired activities that are not captured or detectable in other log data.

Our results further show that while intrusion detection systems successfully trigger alerts for some attack steps, they fail to detect a large fraction of them. We acknowledge that many attack commands closely resemble legitimate administrative actions, and that IDS rule sets intentionally avoid flagging such activities to limit false positives. Nonetheless, this observation highlights the limitations of purely signature-based detection and suggests the need for complementary analysis approaches.

One such complementary approach is LLM-based log interpretation. In our illustrative evaluation, the LLM classified approximately one third of

attack steps with near-perfect accuracy and another third within the top-10 technique predictions. This result is particularly notable given the large search space of 216 MITRE ATT&CK techniques and the zero-shot setting, in which the model is provided with no contextual information beyond the log data generated during each attack step. These findings suggest that LLMs are in fact capable of extracting meaningful semantic patterns from system logs and have the potential to support automated log interpretation, thereby assisting human analysts in understanding attacker behavior.

6.2. Limitations and Threats to Validity

We acknowledge several limitations and threats to validity of our work. Our infrastructure for data collection is limited to Linux-based systems and open-source software, which we deliberately chose to ensure full reproducibility. However, attack manifestations may differ substantially on other operating systems such as Windows, where logging mechanisms, formats, and granularity, vary considerably. As a result, the performance of LLM-based log interpretation may differ significantly. Researchers should therefore complement our data set with Windows-based data sets such as the ones reviewed in Sect. 2.2 for evaluations.

Even though our methodology relies on idle systems and log filtering (cf. Sects. 3.1 and 4.1.1), we cannot guarantee that the collected data sets are entirely free of background noise. At the same time, our filtering procedure may remove events that are influenced by attacks. Some label inaccuracies may be introduced by delayed events that fall outside of their corresponding attack intervals or coincide with other attack steps. To mitigate these issues, we provide both filtered and unfiltered versions of the extracted attack manifestations (cf. Sect. 1). Moreover, despite independent manual validation of attack playbooks and the resulting attack manifestations, we cannot guarantee that every attack step completed as expected since not all steps produce explicit output.

We also emphasize that our evaluation of LLM-based log interpretation is intended as an illustrative case study rather than a comprehensive benchmark. We consider only a single LLM and acknowledge that the observed results may not generalize to other models. Furthermore, limiting the prompt to ten randomly sampled events per log file means that the most informative events may not always be included, thereby affecting classification accuracy. Moreover, our evaluation strategy that counts correct classifications if any

ground truth technique appears in the predicted list may bias results in favor of attack steps with many assigned techniques. For instance, system vulnerability scanning (Scenario 1, Step 13) is associated with 13 techniques, which increases the likelihood of obtaining at least one match in comparison to most attack steps which only involve one or two associated techniques.

Finally, our attack scenarios are not derived from real-world incidents but are intentionally constructed to combine as many attack techniques as practically feasible into coherent kill chains. While scenario design was carried out by domain experts with experience in offensive technologies and attack modeling, these sequences may not fully reflect real attacker behavior. In addition, despite independent cross-validation, the fact that the same team designed and labeled the scenarios may have introduced unintentional bias.

6.3. Future Work

Since the modeled attack cases are labeled at the technique level, the presented data set is directly applicable for evaluation of log interpretation approaches as demonstrated in our case study. Extending these labels could prove useful for evaluation of additional tasks for LLMs in the security domain, including log summarization [12, 13]. In particular, we assign natural-language descriptions to selected attack steps (cf. Fig. 4), which could serve as alternative or complementary ground truth labels. Future extensions could also incorporate labels related to mitigation strategies and recommended countermeasures, e.g., commands to remove backdoors and return the system into a secure state. We plan to release enriched versions of the data set with extended annotations as part of future publications.

In our illustrative evaluation, we only consider raw log data and IDS alerts as input to the LLM. However, additional contextual information could substantially improve attack interpretation and classification. Specifically, our data set includes system configuration files collected from all hosts that contain details about installed software versions and system settings. Such asset information can provide valuable context that verifies or augments attack interpretations. For example, in the ZoneMinder exploit (Scenario 1, Step 5), the configuration files specify the exact version of the ZoneMinder application[37], which is vulnerable to unauthenticated remote code execution

[37] We deploy ZoneMinder version 1.36.31 in our test environment.

according to public threat intelligence[38]. While the LLM is already capable of correctly interpreting the logs generated as consequences of this exploit (cf. Fig. 23) as command execution via the ZoneMinder web application (cf. Fig. 24a), the combination with asset data and external threat intelligence could further corroborate LLM interpretations, increase prediction confidence, enrich explanations with relevant vulnerability references, and potentially suggest actionable mitigation steps.

Another simplification of our evaluation is that we consider attack steps in isolation. In practice, correlating information across multiple steps would enable reasoning about the attacker's objectives and the overall progression of the kill chain. Instead of submitting independent prompts for each step, future work could incorporate log data or classifications from preceding attack steps as contextual information. This could resolve incorrect interpretations regarding benign or malicious intentions of underlying actions (cf. Fig. 25). For example, commands executed by the attacker that are also commonly carried out by system administrators may appear as benign when considered in isolation, but are more likely to be interpreted correctly when immediately following other clearly malicious activities carried out by the same user or within the same context.

Furthermore, we use only a single general-purpose LLM in this paper but recommend to compare performance across different models and architectures in future evaluations. In particular, prior work has shown that LLM fine-tuning can improve log interpretation capabilities [53]. Aside from that, given the already promising results obtained with a general-purpose model, it would be interesting to investigate whether smaller and cost-efficient models can achieve comparable performance [7]. Specifically, locally deployable models could address privacy concerns that potentially prohibit organizations from transferring log data to external LLM providers [15]. We also encourage future studies to compare LLM-based log interpretation with human expert analysis under controlled conditions using identical prompts and data. Such comparisons would provide a more grounded assessment of the practical utility of LLMs in security operations.

[38]All ZoneMinder versions prior to 1.36.33 and 1.37.33 are vulnerable to the exploit considered in Scenario 1: `https://nvd.nist.gov/vuln/detail/CVE-2023-26035`.

7. Conclusion

In this paper, we introduced CAM-LDS, a publicly available data set of cyber attack manifestations in system log data and security alerts. The data set comprises seven coherent attack scenarios covering 81 distinct MITRE ATT&CK techniques across 13 tactics and is specifically designed to support reproducible research in LLM-driven interpretation. By focusing on the direct consequences of attack activity in a controlled and reproducible environment, CAM-LDS enables systematic investigation of how adversarial behavior manifests in Linux-centric system logs.

Our analysis revealed that attack manifestations vary substantially in their characteristics. While many attack steps leave explicit command-level traces in logs, others produce indirect indicators such as anomalous event frequencies, changes in system performance metrics, or intrusion detection alerts. At the same time, a considerable fraction of attack steps remain undetected by signature-based IDS, highlighting the limitations of rule-based detection approaches and the importance of complementary analysis methods such as LLMs.

To illustrate the utility of CAM-LDS, we conducted a zero-shot evaluation of LLM-based log interpretation. The results show that LLMs are capable of correctly identifying the underlying attack technique for a substantial portion of attack steps using raw log data alone. Classification performance appears connected to manifestation characteristics; in particular, observable attack commands, high log frequencies, and the presence of intrusion alerts have positive impacts on classification accuracy.

We release CAM-LDS together with the open-source infrastructure, attack scripts, and experimental artifacts to facilitate transparent, reproducible, and comparable research in the area of log-based security analysis. In particular, we encourage future work to incorporate additional contextual and asset information, as well as knowledge from previously classified attack steps, to further improve classification accuracy and more reliably distinguish malicious activity from benign behavior.

Acknowledgments

Funded by the European Union under the Horizon Europe Research and Innovation programme (GA no. 101168144 - MIRANDA). Views and opinions expressed are however those of the author(s) only and do not necessarily

reflect those of the European Union or the European Commission. Neither the European Union nor the granting authority can be held responsible for them.

References

[1] A. Alzu'bi, O. Darwish, A. Albashayreh, Y. Tashtoush, Cyberattack event logs classification using deep learning with semantic feature analysis, Computers & Security 150 (2025) 104222.

[2] J. Qi, S. Huang, Z. Luan, S. Yang, C. Fung, H. Yang, D. Qian, J. Shang, Z. Xiao, Z. Wu, Loggpt: Exploring chatgpt for log-based anomaly detection, in: 2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), IEEE, 2023, pp. 273–280.

[3] Z. Zhang, S. Li, L. Zhang, J. Ye, C. Hu, L. Yan, Llm-lade: Large language model-based log anomaly detection with explanation, Knowledge-Based Systems 326 (2025) 114064.

[4] S. S. Shanto, R. Paul, Z. Ahmed, A. S. Reza, K. M. Islam, S. S. Roy, Console log explainer: A framework for generating automated explanations using llm, in: 2024 2nd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings), IEEE, 2024, pp. 1–5.

[5] A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman, Survey of intrusion detection systems: techniques, datasets and challenges, Cybersecurity 2 (1) (2019) 1–22.

[6] Y. Liu, S. Tao, W. Meng, F. Yao, X. Zhao, H. Yang, Logprompt: Prompt engineering towards zero-shot and interpretable log analysis, in: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, 2024, pp. 364–365.

[7] G. Palma, G. Cecchi, M. Caronna, A. Rizzo, Leveraging large language models for scalable and explainable cybersecurity log analysis, Journal of Cybersecurity and Privacy 5 (3) (2025) 55.

[8] S. Hans, S. Marsella, S. Hirschmann, N. Gurney, Security logs to att&ck insights: Leveraging llms for high-level threat understanding and cognitive trait inference, arXiv preprint arXiv:2510.20930 (2025).

[9] R. Singh, S. Tariq, F. Jalalvand, M. B. Chhetri, S. Nepal, C. Paris, M. Lochner, Llms in the soc: An empirical study of human-ai collaboration in security operations centres, arXiv preprint arXiv:2508.18947 (2025).

[10] E. Karlsen, X. Luo, N. Zincir-Heywood, M. Heywood, Benchmarking large language models for log analysis, security, and interpretation, Journal of Network and Systems Management 32 (3) (2024) 59.

[11] Y. Liu, S. Tao, W. Meng, J. Wang, W. Ma, Y. Chen, Y. Zhao, H. Yang, Y. Jiang, Interpretable online log analysis using large language models with prompt strategies, in: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, 2024, pp. 35–46.

[12] P. Mudgal, R. Wouhaybi, An assessment of chatgpt on log data, in: International Conference on AI-generated Content, Springer, 2023, pp. 148–169.

[13] Y. Liu, Y. Ji, S. Tao, M. He, W. Meng, S. Zhang, Y. Sun, Y. Xie, B. Chen, H. Yang, Loglm: From task-based to instruction-based automated log analysis, in: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, 2025, pp. 401–412.

[14] Š. Balogh, M. Mlynček, O. Vraňák, P. Zajac, Using generative ai models to support cybersecurity analysts, Electronics 13 (23) (2024) 4718.

[15] A. Habibzadeh, F. Feyzi, R. E. Atani, Large language models for security operations centers: A comprehensive survey, arXiv preprint arXiv:2509.10858 (2025).

[16] L. Cotti, I. Drago, A. Rula, D. Bianchini, F. Cerutti, Ontologx: Ontology-guided knowledge graph extraction from cybersecurity logs with large language models, arXiv preprint arXiv:2510.01409 (2025).

[17] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, ICISSP 1 (2018) 108–116.

[18] S. S. Bagui, D. Mink, S. C. Bagui, T. Ghosh, R. Plenkers, T. McElroy, S. Dulaney, S. Shabanali, Introducing uwf-zeekdata22: A comprehensive network traffic dataset based on the mitre att&ck framework, Data 8 (1) (2023) 18.

[19] M. Elam, D. Mink, S. S. Bagui, R. Plenkers, S. C. Bagui, Introducing uwf-zeekdata24: An enterprise mitre att&ck labeled network attack traffic dataset for machine learning/ai, Data 10 (5) (2025) 59.

[20] GitHub, Mordor, `https://github.com/UraSecTeam/mordor`, accessed February 11, 2026 (2019).

[21] GitHub, Evtx-attack-samples, `https://github.com/sbousseaden/EVTX-ATTACK-SAMPLES`, accessed February 11, 2026 (2020).

[22] GitHub, Evtx-to-mitre-attack, `https://github.com/mdecrevoisier/EVTX-to-MITRE-Attack`, accessed February 11, 2026 (2024).

[23] GitHub, atomic-evtx, `https://github.com/arniki/atomic-evtx`, accessed February 11, 2026 (2025).

[24] A. Pérez-Sánchez, R. Palacios, G. L. López, Comiset: Dataset for the analysis of malicious events in windows systems, Data in Brief (2025) 111723.

[25] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger, A. Rauber, Maintainable log datasets for evaluation of intrusion detection systems, IEEE Transactions on Dependable and Secure Computing 20 (4) (2022) 3466–3482.

[26] D. Sun, J. Zhang, J. Xu, Y. Zheng, Y. Tian, Z. Li, From alerts to intelligence: A novel llm-aided framework for host-based intrusion detection, arXiv preprint arXiv:2507.10873 (2025).

[27] J. J. Tejero-Fernández, A. Sánchez-Macián, Evaluating language models for threat detection in iot security logs, arXiv preprint arXiv:2507.02390 (2025).

[28] J. Zha, X. Shan, J. Lu, J. Zhu, Z. Liu, Leveraging large language models for efficient alert aggregation in aiops, Electronics 13 (22) (2024) 4425.

[29] A. Oliner, J. Stearley, What supercomputers say: A study of five system logs, in: 37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07), IEEE, 2007, pp. 575–584.

[30] J. Zhu, S. He, P. He, J. Liu, M. R. Lyu, Loghub: A large collection of system log datasets for ai-driven log analytics, in: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), IEEE, 2023, pp. 355–366.

[31] M. Landauer, F. Skopik, M. Wurzenberger, W. Hotwagner, A. Rauber, Have it your way: Generating customized log datasets with a model-driven simulation testbed, IEEE Transactions on Reliability 70 (1) (2020) 402–415.

[32] R. D. Medenou, V. M. C. Mayo, M. G. Balufo, M. P. Castrillo, F. J. G. Garrido, A. L. Martinez, D. N. Catalán, A. Hu, D. S. Rodríguez-Bermejo, J. M. Vidal, et al., Cysas-s3: a novel dataset for validating cyber situational awareness related tools for supporting military operations, in: Proceedings of the 15th International Conference on Availability, Reliability and Security, 2020, pp. 1–9.

[33] S. Myneni, A. Chowdhary, A. Sabur, S. Sengupta, G. Agrawal, D. Huang, M. Kang, Dapt 2020-constructing a benchmark dataset for advanced persistent threats, in: International workshop on deployable machine learning for security defense, Springer, 2020, pp. 138–163.

[34] A. Berady, M. Jaume, V. V. T. Tong, G. Guette, Pwnjutsu: A dataset and a semantics-driven approach to retrace attack campaigns, IEEE Transactions on Network and Service Management 19 (4) (2022) 5252–5264.

[35] M. Landauer, F. Skopik, M. Wurzenberger, Introducing a new alert data set for multi-step attack analysis, in: Proceedings of the 17th Cyber Security Experimentation and Test Workshop, 2024, pp. 41–53.

[36] S. Myneni, K. Jha, A. Sabur, G. Agrawal, Y. Deng, A. Chowdhary, D. Huang, Unraveled—a semi-synthetic dataset for advanced persistent threats, Computer Networks 227 (2023) 109688.

[37] C. Laprade, B. Bowman, H. H. Huang, Picodomain: a compact high-fidelity cybersecurity dataset, arXiv preprint arXiv:2008.09192 (2020).

[38] A. Mabrouk, M. Hatem, M. Mamun, S. Saad, Lmdg: Advancing lateral movement detection through high-fidelity dataset generation, arXiv preprint arXiv:2508.02942 (2025).

[39] K. Rahal, A. Riahi, T. Debatty, Dataset of apt persistence techniques on windows platforms mapped to the mitre att&ck framework, in: 2025 28th Conference on Innovation in Clouds, Internet and Networks (ICIN), IEEE, 2025, pp. 17–24.

[40] A. Riddle, K. Westfall, A. Bates, Atlasv2: Atlas attack engagements, version 2, arXiv preprint arXiv:2401.01341 (2023).

[41] S. S. Karim, M. Afzal, W. Iqbal, D. Al Abri, Advanced persistent threat (apt) and intrusion detection evaluation dataset for linux systems 2024, Data in Brief 54 (2024) 110290.

[42] S. Kilian, V. V. T. Tong, J.-F. Lalande, F. Majorczyk, A. Sanchez, N. Talon, P.-V. Besson, H. Orsini, P. Lledo, P.-F. Gimenez, Casinolimit: An offensive dataset labeled with mitre att&ck techniques, in: The 28th International Symposium on Research in Attacks, Intrusions and Defenses (RAID), 2025.

[43] M. Wolf, K. Bergner, P. Förtsch, D. Landes, Implications of applying emulation data to machine learning-based intrusion detection systems, Journal of Information Security and Applications 97 (2026) 104362.

[44] P. Bönninghausen, R. Uetz, M. Henze, Introducing a comprehensive, continuous, and collaborative survey of intrusion detection datasets, in: Proceedings of the 17th Cyber Security Experimentation and Test Workshop, 2024, pp. 34–40.

[45] P. Goldschmidt, D. Chudá, Network intrusion datasets: a survey, limitations, and recommendations, Computers & Security (2025) 104510.

[46] X. Shen, Z. Li, G. Burleigh, L. Wang, Y. Chen, Decoding the mitre engenuity att&ck enterprise evaluation: An analysis of edr performance in real-world environments, in: Proceedings of the 19th ACM Asia conference on computer and communications security, 2024, pp. 96–111.

[47] Information technology - Security techniques - Network security - Part 3: Reference networking scenarios - Threats, design techniques and control issues, Standard, International Organization for Standardization (Dec. 2010).

[48] M. Landauer, W. Hotwagner, T. Boenke, F. Skopik, M. Wurzenberger, Attackmate: Realistic emulation and automation of cyber attack scenarios across the kill chain, arXiv preprint arXiv:2601.14108 (2026).

[49] J. Stühn, J.-N. Hilgert, M. Lambertz, The hidden threat: Analysis of linux rootkit techniques and limitations of current detection tools, Digital Threats: Research and Practice 5 (3) (2024) 1–24.

[50] M. R. Rahman, S. K. Basak, R. M. Hezaveh, L. Williams, Attackers reveal their arsenal: An investigation of adversarial techniques in cti reports, arXiv preprint arXiv:2401.01865 (2024).

[51] S. He, J. Zhu, P. He, M. R. Lyu, Experience report: System log analysis for anomaly detection, in: 2016 IEEE 27th international symposium on software reliability engineering (ISSRE), IEEE, 2016, pp. 207–218.

[52] M. Landauer, M. Wurzenberger, F. Skopik, W. Hotwagner, G. Höld, Aminer: A modular log data analysis pipeline for anomaly-based intrusion detection, Digital Threats: Research and Practice 4 (1) (2023) 1–16.

[53] Y. Ji, Y. Liu, F. Yao, M. He, S. Tao, X. Zhao, C. Su, X. Yang, W. Meng, Y. Xie, et al., Adapting large language models to log analysis with interpretable domain knowledge, in: Proceedings of the 34th ACM International Conference on Information and Knowledge Management, 2025, pp. 1135–1144.